DSWS · 电商问数智能体

设计文档

32 张图 + 每张图的读图地图

读者：评审者、实现者、后续维护者

内容：系统边界、运行时、数据、安全、部署、测试

版本：2026-11-09

一图一页 · 一图一地图

地图目录

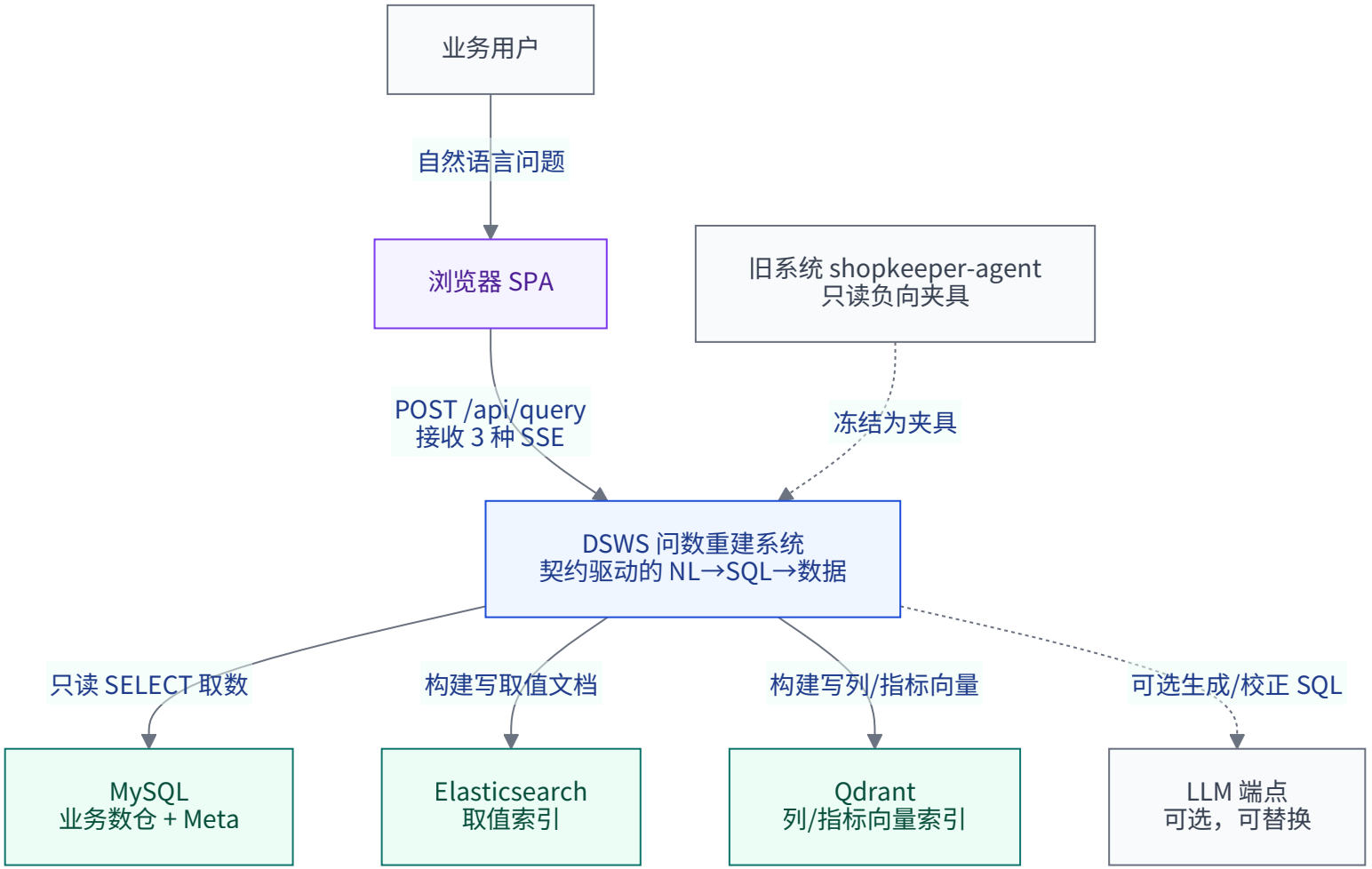
先读这张表：每张图回答一个问题，按 3 步读。正文按图号顺序排列。

#	图与它回答的问题	读图 1→2→3
01	**C4-L1 系统上下文** DSWS 的系统边界在哪？哪些是外部依赖？生产问数主链路到底读哪几个外部系统？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
02	**C4-L2 容器图** 一次请求跨哪些容器或进程？S2 构建与 init 在容器拓扑里处于什么位置？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
03	**backend/app 模块依赖** 谁 import 谁？哪些依赖只发生在构建时，哪些发生在每次请求？安全层和生成层各自依赖什么？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
04	**前端文件树** 前端有哪些文件、谁调用谁？SSE 解析与契约类型分别住在哪？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
05	**阶段演进 S0→S8** 为什么按 S0–S8 顺序重建？每个阶段到底在验收哪一种「还不确定」？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
06	**构建时 vs 查询时** S2 退出之后，查询时每个阶段各自直接读什么？有没有常驻构建进程？	1 先看系统边界与参与者 → 2 再沿主箭头看模块/容器调用 → 3 最后看灰色边与单点：哪些不在主链路
07	**端到端请求生命周期** 一次问数里谁把什么交给谁？失败、空结果、取消分别在哪一步分叉？为什么这条流一定会结束？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
08	**S2 构建流水线时序** 谁在读契约？谁在写三个存储？写序为什么是这个顺序？四个开关各改变了什么？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
09	**S3 召回流水线时序** 三存储各自被问了几次？候选怎么合并？哪些字段会被丢掉？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
10	**S4 生成 SQL 流水线** 上下文怎么进提示词？生成器有两种，怎么选？为什么形状校验必须在静态校验之前？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
11	**S5 安全校验与执行** 策略在哪两个地方各查一次？被拒绝时会不会连库？超时是怎么产生的？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
12	**S6 校正闭环状态机** 什么情况下会走 `correct_sql`？走完之后为什么必须重新校验？哪些失败是不可重试的？	1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边
13		1 先看触发者（谁发起） → 2 沿编号/箭头走一遍主路径 → 3 最后看失败分支与回退边

#	图与它回答的问题	读图 1→2→3
	S7 SSE 事件序列 一条 `progress` 到哪个组件？`error.class` 三个取值各长什么样？取消为什么不算 done?	
14	**数仓 ER 图（5 表）** 每张表一行代表什么？主键是什么？事实表如何关联维度？哪些列是真实存在、哪些是常见误问（`city`、`price`）？	1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
15	**契约模型（表 / 列 / 指标 / 值域 / 列—指标）** 契约里到底有哪些对象？每个对象带哪些字段？`ValueDomain` 的 `kind` 有哪几种？`ColumnMetric` 为什么是派生表?	1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
16	**一份契约如何落到三存储（region_name / order_amount）** `region_name` 和 `order_amount` 分别变成什么？为什么一个进 ES、一个不进？Qdrant 的 98 个 point 是怎么来的？Meta 的 `examples` 能不能当全集?	1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
17	**S2 数据 lineage（契约 + 冻结数仓 → 三存储 + manifest）** 数据从哪来？在哪一步校验？三存储各写多少？`5/24/6/9/76/98/22` 分别是什么？manifest 为什么是构建成功的唯一凭据?	1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
18	**Meta 表结构（5 张元数据表）** 每张 Meta 表的字段和主键是什么？`column_info` 怎么指向表？`column_metric` 的复合主键是什么？manifest 记录哪些计数?	1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
19	**值域分类决策树（哪些列写 ES）** `enum` 和 `range` 分别怎么处理？`open` 文本为什么有的进 ES 有的不进？`sync_values` 到底记什么？Meta 的 `examples` 和 ES 的完整值域是什么关系?	1 先看实体与 grain（一行代表什么） → 2 再看主键/外键与关系 → 3 最后看约束、值域与口径
20	**信任边界与攻击面** 一次问数跨过哪些信任边界？谁持有数据库凭据？LLM 能拿到什么、拿不到什么？旧系统会不会被新系统写入?	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
21	**五层纵深防御** 一条 SQL 从模型出来到数据库要过几关？每关挡住什么？为什么说「策略被绕过也还有数据库权限兜底」？失败最终怎么到 UI?	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
22	**SQL 策略判定流程（六关 + 收尾）** 每一关拒绝什么？拒绝后会不会连库？为什么执行的不是模型返回的原文？guard 和 policy 的先后顺序是什么?	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
23	**只读账号权限模型（dw_reader 与应用账号分离）** `dw_reader` 是谁、怎么创建、怎么验证？执行器和构建任务用的是不是同一个账号？旧 `GRANT ALL` 为什么危险？应用账号与只读账号怎么分离?	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
24	**错误码分类（16 码 / 3 class / retryable）** 16 个错误码分别属于哪个 class？哪些能自动校正？空结果和用户取消为什么不算失败？失败最后怎么变成 SSE 事件?	1 先看信任边界在哪 → 2 再按顺序看拦截点 → 3 最后看被拒绝的路径与出口
25	**Docker Compose 拓扑** 哪六个服务？谁依赖谁？数据落在哪？`init` 为什么和另外五个不一样?	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点
26	**宿主端口与容器端口暴露面** 谁能从局域网进来？哪些端口只能本机访问？容器之间走什么端口?	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点

#	图与它回答的问题	读图 1→2→3
27	**启动顺序与失败停点** 谁在等谁？每级等多久？失败会停在哪一步？`init` 内部又做了什么？	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点
28	**旧系统与 rebuild 的隔离边界** 旧系统的 `meta` / `value_index` / `5173` / `3306` 与 rebuild 的 `*_rebuild` / `5273` / `8100` 在哪一层分开？哪些东西其实是共用的？	1 先看服务与端口 → 2 再看启动依赖顺序 → 3 最后看健康检查与失败停点
29	**测试金字塔与三存储依赖** 五层测试各自覆盖什么？哪一层开始需要 MySQL / ES / Qdrant？哪些是「本机就能重复」的门禁？	1 先看层级（单元/集成/端到端） → 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码
30	**S0–S7 阶段门禁** 每个阶段跑什么？「通过」的判据是什么？哪两个阶段需要先起三存储？	1 先看层级（单元/集成/端到端） → 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码
31	**失败注入矩阵** 失败从哪里注入？落到哪个错误码？属于 `failure` / `empty` / `cancelled` 哪一类？哪些会触发校正闭环？界面上长什么样？	1 先看层级（单元/集成/端到端） → 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码
32	**证据台账：已验证 / 推断 / 未验证** 图里的结论从哪来？哪些能在本机复现？哪些只是引用他机记录？最终汇到哪里？	1 先看层级（单元/集成/端到端） → 2 再看阶段门禁顺序 → 3 最后看失败注入覆盖了哪些码

图 01 C4-L1 系统上下文



架构 · 读图说明见下一页

图 01 · 架构

C4-L1 系统上下文

> DSWS 是契约驱动的问数系统，用户经浏览器提交问题，系统只对 MySQL 做只读取数；ES 与 Qdrant 由 S2 写入、仅由 S3 阶段验收读取，LLM 是可替换的可选项。

这张图回答：DSWS 的系统边界在哪？哪些是外部依赖？生产问数主链路到底读哪几个外部系统？

读图 1→2→3

1. 先看系统边界与参与者
2. 再沿主箭头看模块/容器调用
3. 最后看灰色边与单点：哪些不在主链路

失败：箭头断开或灰色边表示不在主链路，不是故障；真正的失败回到对应章节看错误码。

图例：灰=人/外部系统；紫=运行时前端；蓝=DSWS 系统边界；青=存储；虚线=可选或只读关系。

关键元素：

- **业务用户 → 浏览器 SPA**：输入自然语言问题。
- **浏览器 → DSWS**：`POST /api/query`，接收 3 种 SSE 事件。
- **DSWS → MySQL**：只读 `SELECT` 取业务数据与 Meta。
- **DSWS → ES / Qdrant**：S2 构建时写取值文档与列/指标向量；S3 阶段验收时读取，生产问数主链路不读。
- **DSWS ⇢ LLM**：可选生成/校正；确定性 provider 可完全替代。
- **旧系统 ⇢ DSWS**：`shopkeeper-agent` 冻结为只读负向夹具，不参与运行。

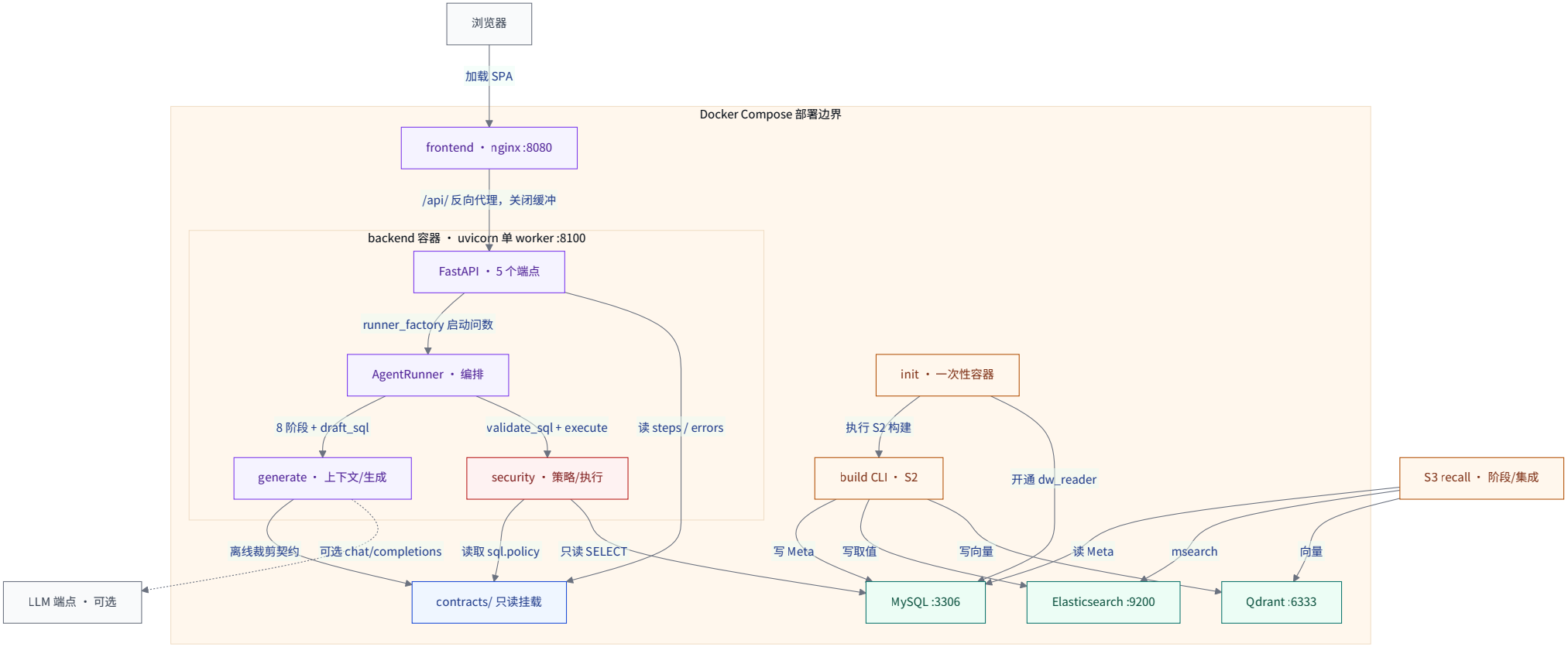
证据：

- `backend/app/main.py` L147 `create_app`，L170–L236 共 5 个端点（✅文件存在）
- `contracts/` 下 13 个契约文件（`steps.yaml`、`errors.yaml`、`sse.schema.json` 等，✅本机 `ls`）
- `README.md` L195：上下文 8 阶段，契约离线裁剪，S3 召回可选
- `README.md` L232–L238：S2 只写 `*_rebuild` 目标；生产问数上下文由 `OfflineContextBuilder` 生成，S3 的 ES/Qdrant 不在主链路
- `backend/app/generate/provider.py`：确定性/HTTP 双实现（✅文件存在）
- `fixtures/` 下旧系统相关夹具（✅本机 `ls`）

边界：

- C4-L1 不展开容器内部；容器职责见 `02-c4-container.md`。
- 图中用一条边概括「S2 写 / S3 读」三存储；构建与查询的时序关系见 `06-build-vs-query.md`。
- 旧系统未运行，只有文件级夹具（🟡夹具内容已读，运行行为未复现）。
- LLM 外部端点未联调（⚪缺少可用端点与凭据）。

图 02 C4-L2 容器图



架构 · 读图说明见下一页

图 02 · 架构

C4-L2 容器图

> 一次请求依次穿过浏览器 → nginx :8080 → FastAPI :8100 → AgentRunner → generate/security。S2 构建由 init 一次性容器调用 build CLI，写完三存储后退出。

这张图回答：一次请求跨哪些容器或进程？S2 构建与 init 在容器拓扑里处于什么位置？

读图 1→2→3

1. 先看系统边界与参与者
2. 再沿主箭头看模块/容器调用
3. 最后看灰色边与单点：哪些不在主链路

失败：箭头断开或灰色边表示不在主链路，不是故障；真正的失败回到对应章节看错误码。

图例：橙=构建时容器/任务；紫=查询时进程；红=安全；蓝=契约；青=存储；灰=外部。

关键元素：

- **frontend · nginx :8080**：唯一对外入口，`/api/` 反向代理到 backend，关闭响应缓冲。
- **backend · uvicorn 单 worker :8100**：FastAPI 5 个端点，SSE 线程/队列桥接。
- **AgentRunner**：编排 12 步；生成、校验、校正、执行、取消。
- **generate**：`OfflineContextBuilder` 8 阶段 + 7 槽位提示词；可选 HTTP LLM。
- **security**：AST 策略 + `ReadOnlyExecutor`，只读 MySQL。
- **init · 一次性容器**：执行 S2 构建、开通 `dw_reader`，完成后退出。
- **contracts/**：只读挂载，steps/errors/schema/policy 的唯一事实源。
- **MySQL / ES / Qdrant**：构建写入，查询侧仅 S5 读 MySQL、S3 验收读三者。
- **S3 recall**：画在部署边界外，表示阶段验收/集成测试使用，不在生产请求路径。

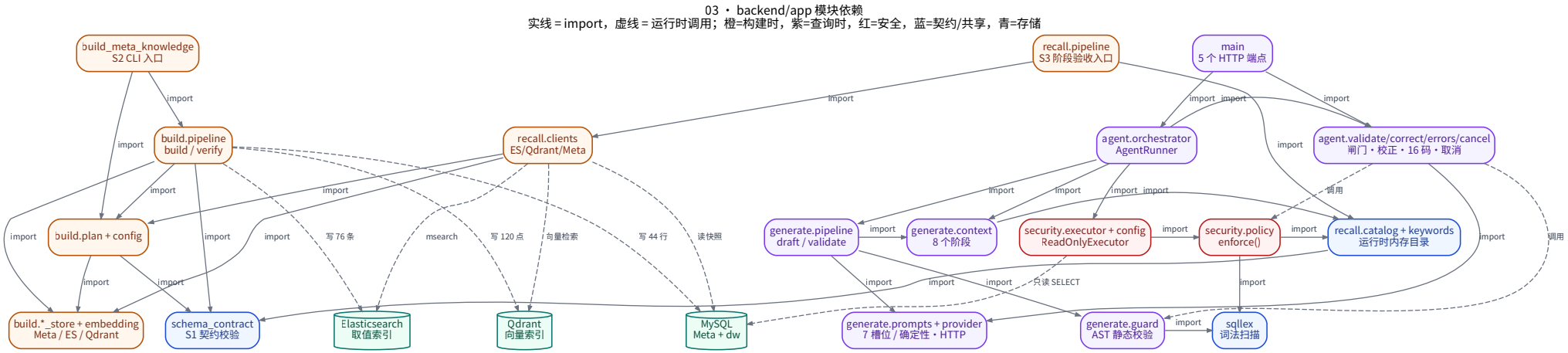
证据：

- `compose.yaml` L11–L160：mysql/elasticsearch/qdrant/init/backend/frontend 六服务与端口
- `backend/app/main.py` L147–L236：`create_app` 与 5 个端点
- `backend/app/agent/orchestrator.py` L126 `AgentRunner`，L199 `events()`
- `backend/app/generate/context.py` L363–L376：8 个上下文阶段
- `backend/app/security/policy.py` L298 `enforce()`；`security/executor.py` L161、L226
- `backend/app/build_meta_knowledge.py` L87–L109：CLI 入口与 4 个开关
- `README.md` L74–L76：init/backend/frontend 端口与职责

边界：

- Compose 是部署形态；本地 `make dev-backend` / `make dev-frontend` 不经 nginx。
- S3 recall 的「边界外」是架构判断（🟡 依据 README 与 orchestrator 导入关系），不是容器缺失。
- LLM 端点可选且未联调（⚪）；确定性 provider 路径可离线验收。
- 图中端口为宿主机映射；容器内端口见 `compose.yaml`。

图 03 backend/app 模块依赖



架构 · 读图说明见下一页

图 03 • 架构

backend/app 模块依赖

> `backend/app` 分成构建时与查询时两条依赖链，共享 `schema_contract`、`sqllex` 和运行时内存目录 `recall.catalog + keywords`。

这张图回答：谁 import 谁？哪些依赖只发生在构建时，哪些发生在每次请求？安全层和生成层各自依赖什么？

读图 1→2→3

1. 先看系统边界与参与者
2. 再沿主箭头看模块/容器调用
3. 最后看灰色边与单点：哪些不在主链路

失败：箭头断开或灰色边表示不在主链路，不是故障；真正的失败回到对应章节看错误码。

图例：实线=import；虚线=运行时调用；橙=构建时；紫=查询时；红=安全；蓝=契约/共享；青=存储。

关键元素：

- **构建时**：`build_meta_knowledge` → `build.pipeline` → `build.plan + config` / `build.*_store` / `build.embedding`；写 MySQL、ES、Qdrant 后退出。
- **查询时**：`main` → `agent.orchestrator` → `agent.validate/correct/errors/cancel`；orchestrator 再调用 `generate.context/pipeline` 与 `security.executor`。
- **共享目录**：`recall.catalog + keywords` 被 generate、security、recall 共同 import，是**内存目录**，不是 S3 检索客户端。
- **生成**：`generate.pipeline` → `context` / `guard` / `prompts + provider`。
- **安全**：`security.policy` → `sqllex` + `catalog`；`executor` → `policy`；executor 只读 MySQL。
- **S3 召回**：`recall.pipeline + nodes` → `recall.clients` 读三存储，仅阶段验收使用。

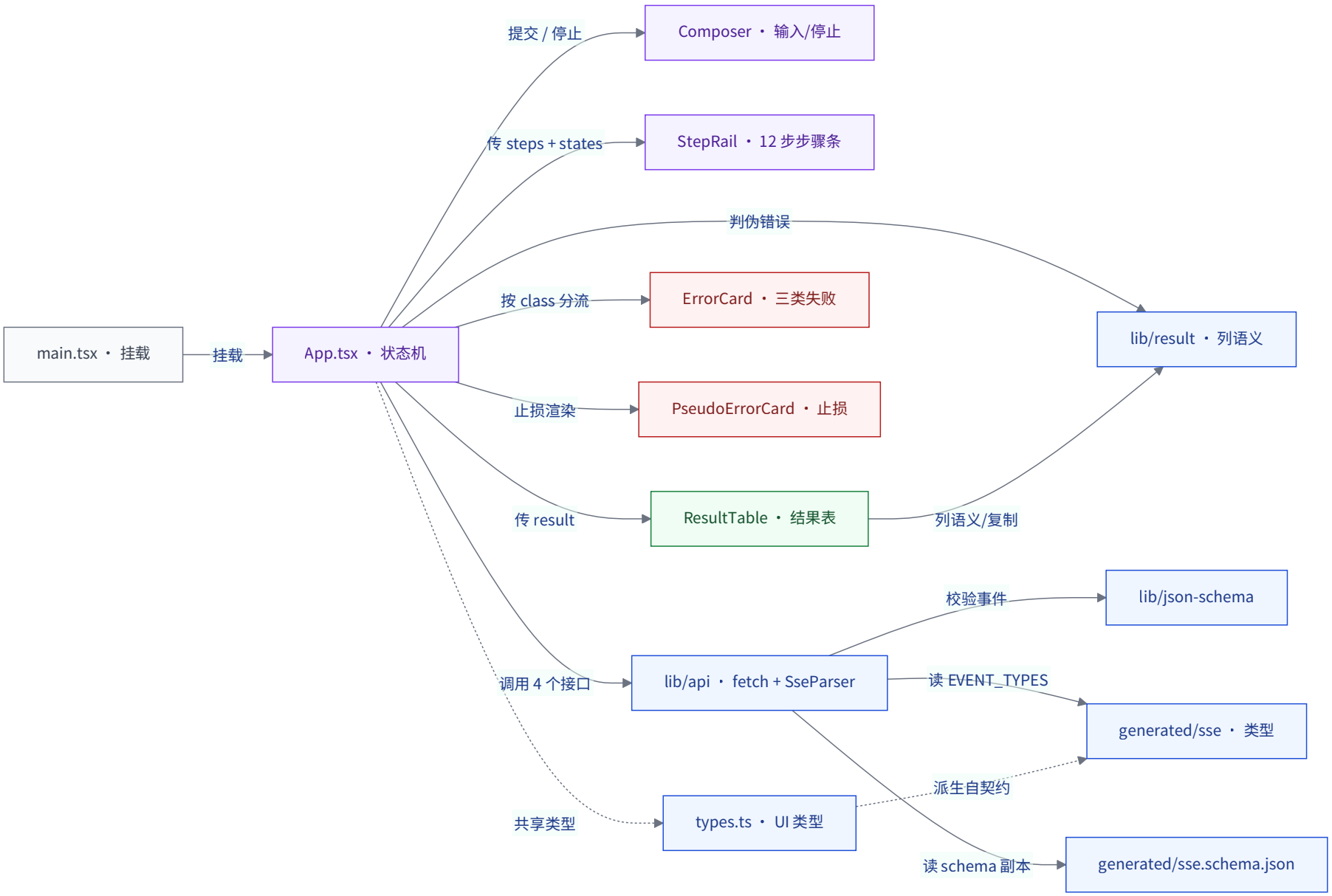
证据：

- 本机 AST 提取的 `app.*` import 清单（✅ 2026-10-29，脚本遍历 `backend/app/**/*.py`）
- `backend/app/generate/context.py` L30–L32：import `recall.catalog` / `recall.keywords` / `recall.models`
- `backend/app/generate/pipeline.py` L192–L206：`generate_with_recall` 备用路径（未画入主图）
- `backend/app/agent/orchestrator.py` L43、L126、L199–L214
- `backend/app/security/policy.py` L298；`security/executor.py` L161、L226
- `backend/app/recall/pipeline.py` L35；`recall/nodes.py` L260–L261
- `backend/app/build/pipeline.py`、`build/plan.py`、`build/*_store.py`（✅文件存在）

边界：

- 图把文件聚合成文件组；逐文件职责与入口见 `inventory/software-census.md` §3。
- `generate_with_recall` 是备用完整召回路径，生产主链路用 `build_offline_context`，因此没有画 `generate.pipeline → recall.pipeline` 的边。
- 实线 import 由 AST 实测（✅）；`validate → guard/policy` 的虚线「调用」由代码阅读推断（🟡）。
- 脚本（`scripts/*`）与 `scenarios/` 未入图，避免外围节点淹没主依赖；其清单见 census §3.7。

图 04 前端文件树



架构 · 读图说明见下一页

图 04 · 架构

前端文件树

> `App.tsx` 是唯一状态机，向下分发到 5 个组件；网络请求与 SSE 解析都在 `lib/api.ts` 的 `SseParser` 里。

这张图回答：前端有哪些文件、谁调用谁？SSE 解析与契约类型分别住在哪？

读图 1→2→3

1. 先看系统边界与参与者
2. 再沿主箭头看模块/容器调用
3. 最后看灰色边与单点：哪些不在主链路

失败：箭头断开或灰色边表示不在主链路，不是故障；真正的失败回到对应章节看错误码。

图例：紫=页面/组件；蓝=库与契约生成物；青=结果表；灰=入口。

关键元素：

- **main.tsx**：挂载 `App`。
- **App.tsx · 状态机**：`switch (event.type)` 处理 progress/result/error；`stop()` 调 `cancelQuery`；判定伪错误后分流。
- **Composer**：输入与停止按钮。
- **StepRail**：按 `steps.yaml` 渲染 12 步状态条。
- **ResultTable**：结果表；列语义与复制载荷来自 `lib/result`。
- **ErrorCard**：按 `class` 展示三类失败。
- **PseudoErrorCard**：把「成功但像错误」的结果单独展示。
- **lib/api.ts**：`fetchSteps` / `fetchErrorCodes` / `streamQuery` / `cancelQuery`，以及 `SseParser` 与 `parseAgentEvent`。
- **lib/result.ts**：`describeColumns` / `buildCopyPayload` / `looksLikePseudoError`。
- **lib/json-schema.ts**：轻量 schema 校验。
- **generated/**：`sse.ts`（类型 + `EVENT_TYPES`）、`sse.schema.json`、`contract.json`。

证据：

- `frontend/src/App.tsx` L140–L162 事件 `switch`；L238–L241 `stop()`；L336–L343 伪错误分流
- `frontend/src/lib/api.ts` L47 `parseAgentEvent`；L78 `SseParser`；L169/L197/L211/L232 四个 API 函数
- `frontend/src/lib/result.ts` L41 / L76 / L103 三个导出
- `frontend/src/generated/sse.ts` L69 `EVENT_TYPES = ["progress","result","error"]`
- `frontend/src/components/` 下 5 个组件（✅ 本机 `ls`）
- `frontend/src/generated/` 下 3 个生成物（✅ 本机 `ls`）

边界：

- **仓库没有 `frontend/src/lib/sse.ts`**；SSE 解析器实际位于 `lib/api.ts` 的 `SseParser`（✅ 本机 `ls frontend/src/lib`）。本图按真实文件画。
- 前端未做浏览器端到端实跑（⚪）；结构结论来自源码 import 与导出清单。
- 图不展开 `sse.test.ts` / `result.test.ts` 两个测试文件，避免与运行时树混淆。

图 05 阶段演进 S0→S8



架构 • 读图说明见下一页

图 05 · 架构

阶段演进 S0→S8

> S0–S8 每阶段只新增一类不确定性，并给出一个可停点。S0–S2 处理契约与构建，S3–S5 处理数据、生成与安全，S6–S8 处理失败、界面与性能。

这张图回答：为什么按 S0–S8 顺序重建？每个阶段到底在验收哪一种「还不确定」？

读图 1→2→3

1. 先看系统边界与参与者
2. 再沿主箭头看模块/容器调用
3. 最后看灰色边与单点：哪些不在主链路

失败：箭头断开或灰色边表示不在主链路，不是故障；真正的失败回到对应章节看错误码。

图例：蓝=契约；橙=构建时；紫=查询时；红=安全；绿=已验证的界面；灰=未实现（S8）。

关键元素：

- **S0 骨架**：不确定「假流永远成功」→ 可停点「12 步协议回放」。
- **S1 Schema 契约**：不确定「数据语义」→ 可停点「5 表 / 24 列 / 6 指标校验」。
- **S2 元知识**：不确定「三存储漂移」→ 可停点「manifest 对账 NOOP」。
- **S3 检索**：不确定「召回质量」→ 可停点「20 题黄金集」。
- **S4 生成**：不确定「SQL 合法性」→ 可停点「AST 静态校验」。
- **S5 安全**：不确定「恶意 SQL」→ 可停点「36 条注入全拒」。
- **S6 失败通道**：不确定「失败无名字」→ 可停点「16 码 / 6 类失败」。
- **S7 前端**：不确定「界面契约漂移」→ 可停点「14 条 SSE 夹具回放」。
- **S8 性能**：不确定「延迟与预算」→ 可停点「P50/P95 目标」（⚪未实现）。

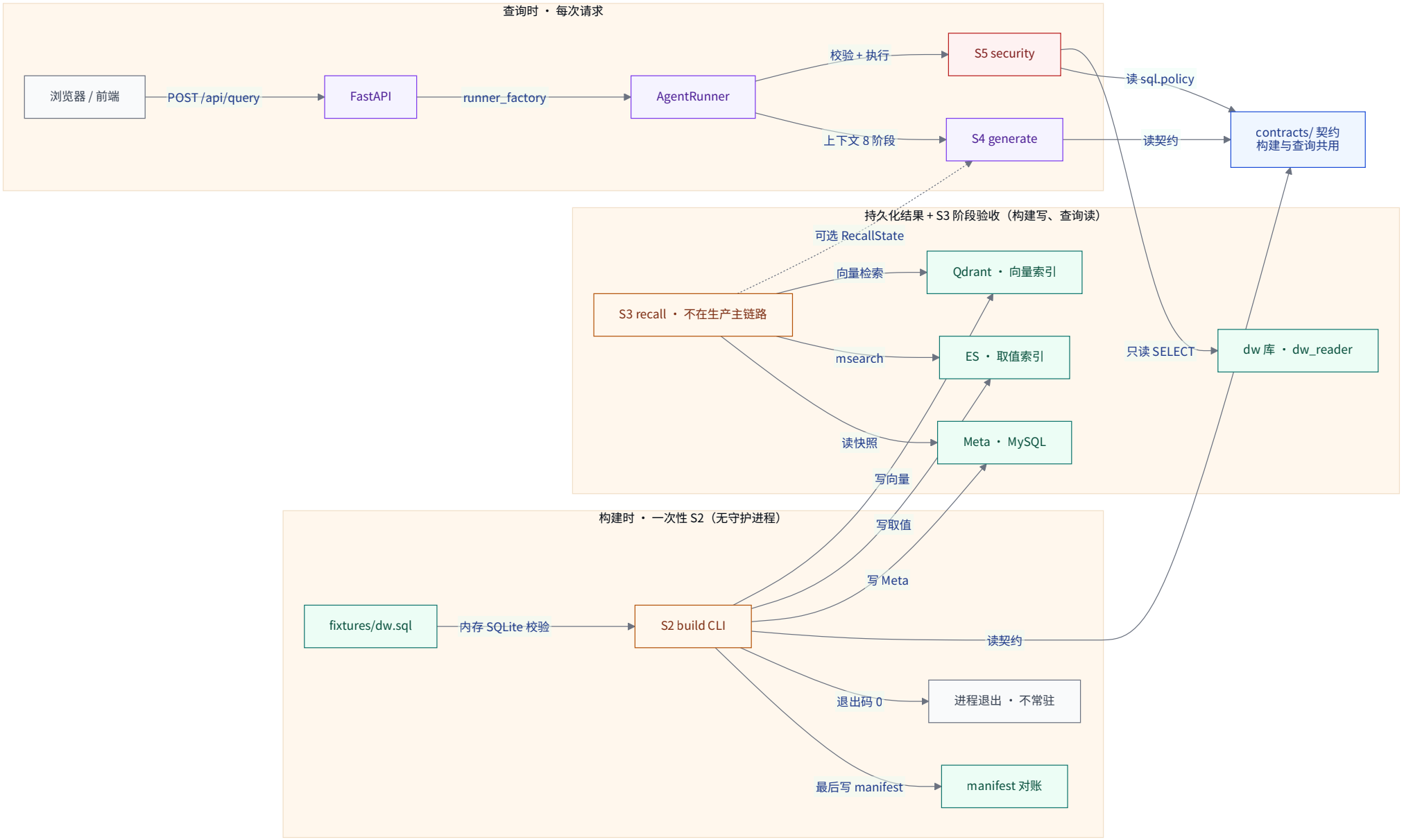
证据：

- `README.md` L13–L24：S0–S8 阶段表与验收命令；L24 明确 S8「待做」
- `Makefile` L1、L14–L156：`stage0` – `stage7` 目标齐全
- `design/WBS.md` L44–L52：图目录中的「阶段演进图 S0–S8」
- `contracts/steps.yaml`：12 步，`correct_sql` 条件执行（✅YAML 解析）
- `contracts/errors.yaml`：16 码 / 3 类（✅YAML 解析）
- `fixtures/injection/sql-injection.yaml`：36 拒 + 11 允许（✅YAML 解析）

边界：

- 每格只写一个「新增不确定性」，该阶段的其他工作见手册章节。
- S8 灰框表示未实现（⚪）；P50/P95 目标值未在本图给出。
- 「可停点」是验收口径，不代表阶段可独立交付；阶段间仍有前置依赖（如 S3 需要 S2 的三存储）。

图 06 构建时 vs 查询时



架构 · 读图说明见下一页

图 06 · 架构

构建时 vs 查询时

> S2 是一次性 CLI，退出后不常驻；查询侧 S4 直接读 contracts、S5 只读 MySQL `dw`，只有阶段验收的 S3 读 ES/Qdrant。

这张图回答：S2 退出之后，查询时每个阶段各自直接读什么？有没有常驻构建进程？

读图 1→2→3

1. 先看系统边界与参与者
2. 再沿主箭头看模块/容器调用
3. 最后看灰色边与单点：哪些不在主链路

失败：箭头断开或灰色边表示不在主链路，不是故障；真正的失败回到对应章节看错误码。

图例：橙=构建时；紫=查询时；青=存储；蓝=契约；红=安全；虚线=可选路径或只读关系。

关键元素：

- **构建时 · S2（无守护进程）**：`fixtures/dw.sql` 做内存 SQLite 校验，`S2 build CLI` 最后写 manifest，进程退出码 0，不常驻。
- **持久化结果 + S3**：Meta/ES/Qdrant 由 S2 写；`S3 recall` 仅阶段验收读取，并可通过 `RecallState` 可选喂给 S4。
- **查询时**：浏览器 → FastAPI → AgentRunner → S4/S5。
- **契约共用**：S2 CLI、S4、S5 都直接读 `contracts/`；S5 另读 `sql.policy`。
- **S5 取数**：`ReadOnlyExecutor` 只读 `dw` 库；S4 不读任何业务存储。
- **S3 位置**：与存储同区，读的是持久化结果；不在生产主链路。

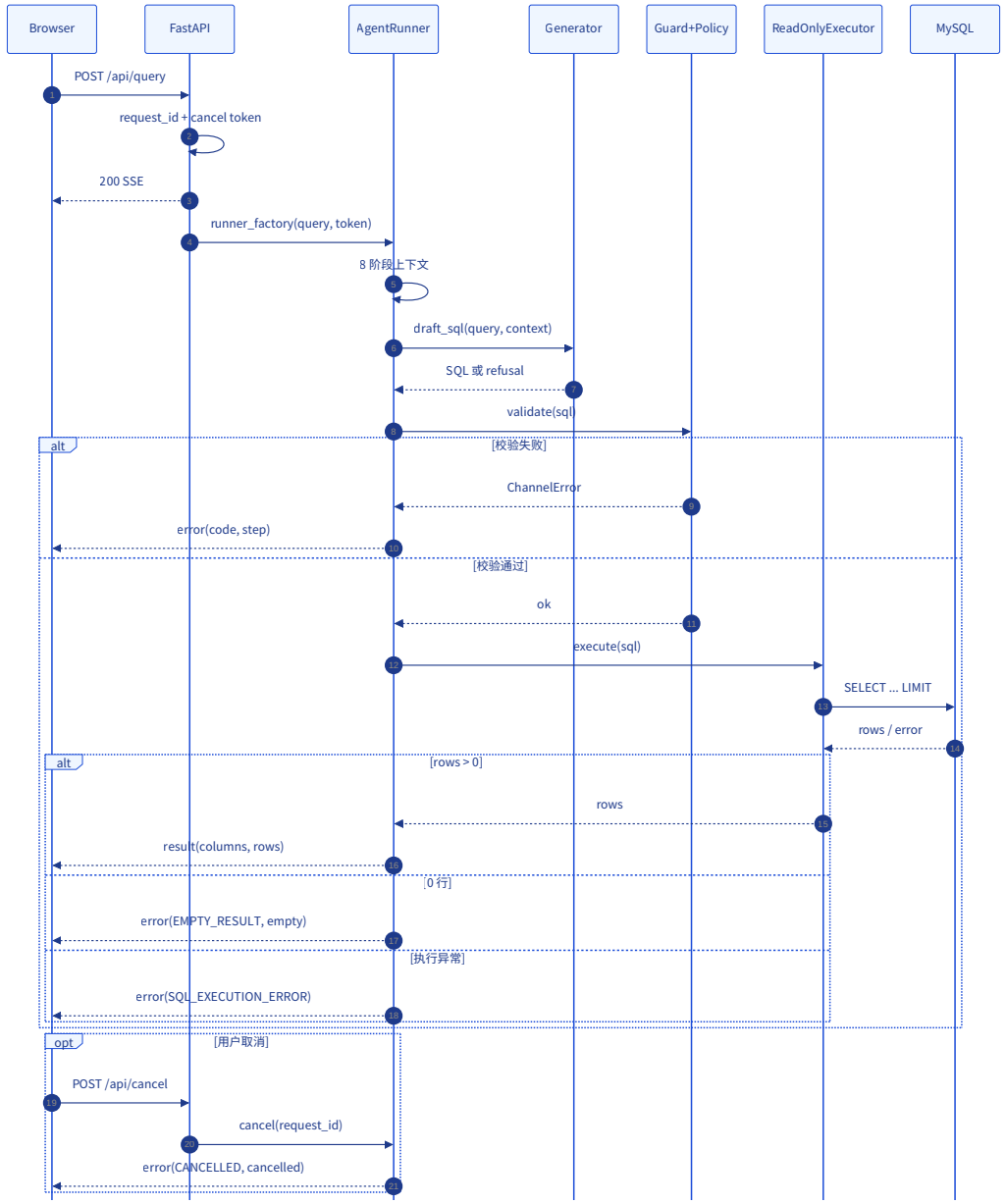
证据：

- `backend/app/build_meta_knowledge.py` L4–L7、L87–L109：`--dry-run/--fresh/--verify/--assert-idempotent`
- `backend/app/generate/context.py` L94 `OfflineContextBuilder`、L392 `build_offline_context`
- `backend/app/agent/orchestrator.py` L43、L214：主链路注入 `OfflineContextBuilder`
- `backend/app/security/executor.py` L161、L226：`ReadOnlyExecutor.execute`
- `backend/app/recall/pipeline.py` L35：`recall()` 入口
- `README.md` L195：上下文 8 阶段，可选 S3 召回
- `README.md` L232–L238：S2 只写 `*_rebuild`；生产问数不读 S3 的 ES/Qdrant
- `compose.yaml` L83–L104：init 一次性容器执行 S2 后退出

边界：

- 图中「S3 → S4」是可选 `RecallState` 路径（🟡 代码存在 `context_from_recall_state`，生产默认不走）。
- S5 的 `dw` 库连接依赖 `dw_reader` 账号；本图不展开授权细节。
- S2 的实际写入计数（44/76/120）来自契约与 manifest 设计；本机未跑 Docker 构建（⚪，见 census §12）。

图 07　端到端请求生命周期



运行时 · 读图说明见下一页

图 07 • 运行时

端到端请求生命周期

> 一次 `POST /api/query` 由 FastAPI 建 `request_id` 与取消开关，交给 `AgentRunner` 在工作线程里跑「上下文 → 生成 → 校验 → 执行」，中间状态逐帧写成 SSE `progress`，最后**必**以 `result` 或 `error` 收尾。

这张图回答：一次问数里谁把什么交给谁？失败、空结果、取消分别在哪一步分叉？为什么这条流一定会结束？

读图 1→2→3

1. 先看触发者（谁发起）
2. 沿编号/箭头走一遍主路径
3. 最后看失败分支与回退边

失败：看红色分支与回退边；没有回退边的步骤失败即终止。

图例：紫色块 = 查询时服务（8 阶段上下文）； `alt` = 分叉； `opt` = 可选路径（用户取消）。

参与者（真实进程 / 类）

图上名字	真实实体	文件
`Browser`	React `App` + `streamQuery`	`frontend/src/App.tsx`、`frontend/src/lib/api.ts` L232
`FastAPI`	`create_app()` → `query()` → `_stream()`	`backend/app/main.py` L147 / L207 / L262
`SSE`	`_sse()`，一行一帧 `data: {...}\n\n`	`backend/app/main.py` L250
`AgentRunner`	`AgentRunner.events()` → `_run()`	`backend/app/agent/orchestrator.py` L126 / L199 / L211
`ContextBuilder`	`OfflineContextBuilder`（8 个阶段）	`backend/app/generate/context.py` L363
`Generator`	`DeterministicGenerator` 或 `HttpLlmGenerator`	`backend/app/generate/provider.py` L138
`Guard`	`validate_sql()` = `check_sql()` + `enforce()`	`backend/app/agent/validate.py`
`ReadOnlyExecutor`	`ReadOnlyExecutor.execute()`	`backend/app/security/executor.py` L161 / L226
`MySQL`	只读账号连接（密码来自 `REBUILD_DW_RO_PASSWORD`）	`backend/app/security/executor.py` L32 / L187

主线（谁在什么时候把什么交给谁）

1. `query()` 生成 `request_id`，在 `CANCEL_REGISTRY` 登记取消开关，把 `X-Request-Id` 放进响应头（`main.py` L211–L226）。前端靠这个头调 `/api/cancel`。
2. `_stream()` 把**同步**的 `runner.events()` 放进工作线程，事件经 `asyncio.Queue` 回到协程，再逐帧 `_sse()`（`main.py` L262–L295）。
3. `AgentRunner._run()` 先跑 8 个上下文阶段，每个阶段**先发 running、再真跑、再发 success + duration_ms**（`orchestrator.py` L222–L237）。

4. `draft_sql()` 拿 `GenerateContext` 生成 SQL 或拒答；`validate_sql()` 过 guard + policy；`_execute()` 走只读执行器。
5. `to_result_payload()` 把 `ExecutionResult` 摊成 `{columns:[{name,type}], rows:[[...]]}`（`orchestrator.py` L86）。

分叉点

结局	分叉位置	产出
成功	执行 `ok` 且 `rows` 非空（L365–L389）	`result` 事件
空结果	执行 `ok` 但 `rows == []`（L373–L388）	`error` + `EMPTY_RESULT`，`class=empty`，**步骤仍是绿的**
失败	校验失败 L304–L306 / 生成拒答 L261–L273 / 执行抛异常 L352–L363 / 执行返回错误 L392–L415	`error`，码取自 `contracts/errors.yaml`
取消	`/api/cancel` → `CancelRegistry.cancel()`（`main.py` L236），或客户端 abort → `_stream` 的 `finally`（L298–L299）	`error` + `CANCELLED`，`class=cancelled`

取消只在**步骤边界**生效：`_cancelled()` 在每个阶段/生成/校验/执行前后各查一次（L219、L286、L342、L367）。已经跑完的步骤不会回滚，后面的步骤不再开始。

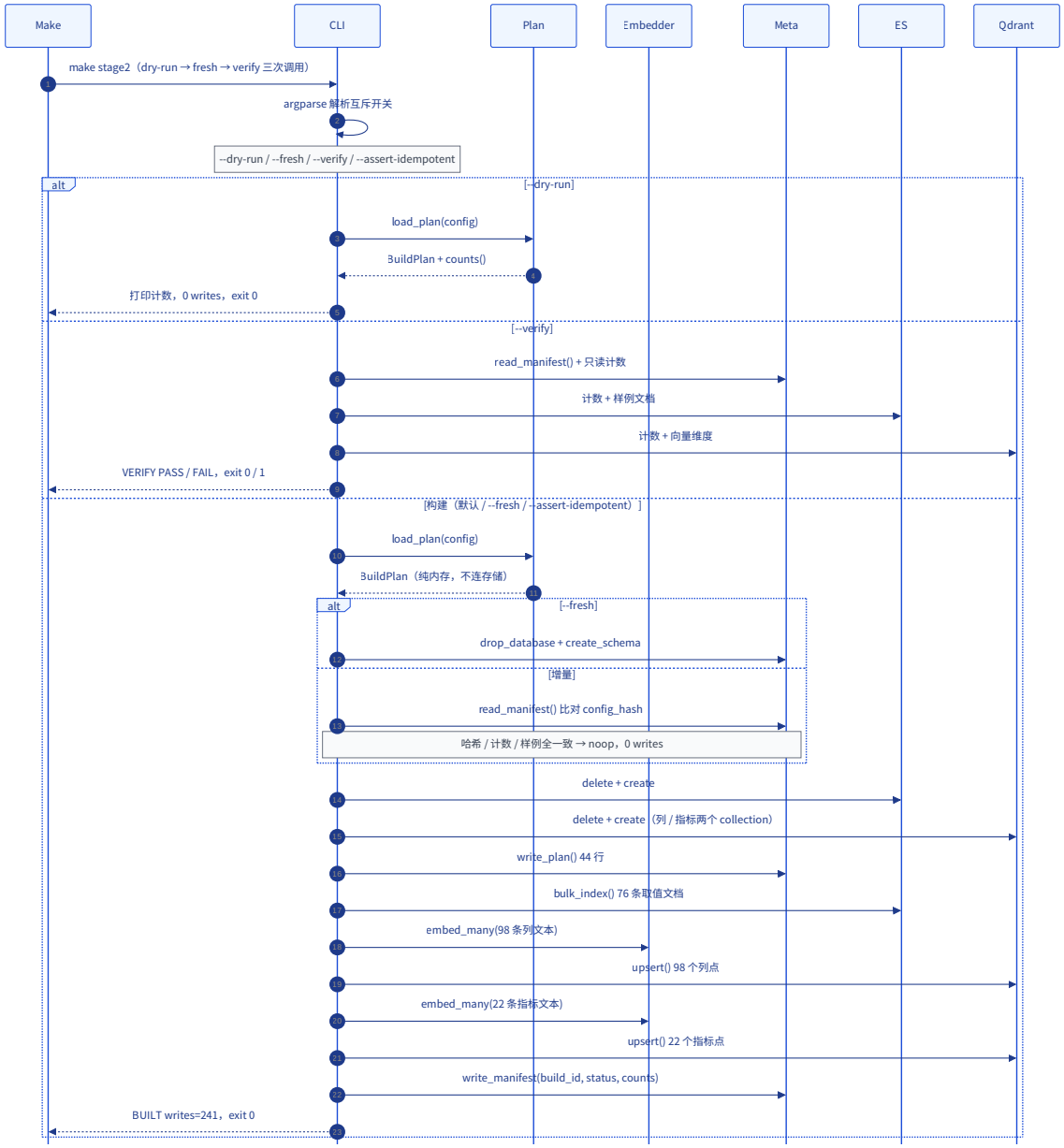
证据

结论	状态	出处
流以 `result` 或 `error` 收尾，异常被兜成 `INTERNAL_ERROR`	✅	`orchestrator.py` L199–L209
8 阶段与 `contracts/steps.yaml` 前 8 项一一对应	✅	`context.py` L363–L376 对照 `steps.yaml`
同步生成器跑在工作线程、经队列回传	✅	`main.py` L272–L294
空结果是 `EMPTY_RESULT` 而不是空 `result`	✅	`orchestrator.py` L373–L388
取消是独立状态、不是 done	✅	`main.py` L227–L237、`orchestrator.py` L181–L187
真实 LLM 生成	⚪	无端点 / 无 Key，本机未跑
浏览器端到端（真实点击 + SSE）	⚪	本次未起前端

边界与已知偏差

- **生产请求路径的 8 阶段是 `OfflineContextBuilder`，不是 `RecallPipeline`**。前者只读内存里的契约目录（`default_catalog()`），**不访问 ES / Qdrant / Meta**；后者（`backend/app/recall/pipeline.py`）才连三存储，只在 S3 CLI 与 `generate_with_recall()` 里用。所以本图没有画 ES / Qdrant。三存储召回见 09。
- `_stream()` 的 `finally` 无条件 `cancel + release`，所以客户端断开时即使工作线程还在跑，也会在下一个边界停下。
- 图中 `error(code, step=validate_sql)` 只画了终止分支；`retryable=true` 时会先插入一次 `correct_sql`，见 12。

图 08 S2 构建流水线时序



运行时 · 读图说明见下一页

图 08 • 运行时

S2 构建流水线时序

> S2 是一条**一次性 CLI 任务**：解析四个互斥开关 → 在内存里生成 `BuildPlan` → 按 Meta → ES → Qdrant → manifest 的顺序写入 → 打印计数并退出，查询时不再运行。

这张图回答：谁在读契约？谁在写三个存储？写序为什么是这个顺序？四个开关各改变了什么？

读图 1→2→3

1. 先看触发者（谁发起）
2. 沿编号/箭头走一遍主路径
3. 最后看失败分支与回退边

失败：看红色分支与回退边；没有回退边的步骤失败即终止。

图例：橙色语义 = 构建时任务（Make / CLI）；青色 = 三个存储；蓝色 = 契约派生的计划。

参与者（真实进程 / 类）

图上名字	真实实体	文件
`Make`	`make stage2`，连续调 CLI 三次	`Make`
`CLI`	`app.build_meta_knowledge.main()`	`backend/app/build_meta_knowledge.py`
`Plan`	`load_plan()` → `BuildPlan.counts()`	`backend/app/build/pla`
`Embedder`	`hash-ngram-v1:256`（确定性，不调外部服务）	`backend/app/build/embedding.py`
`Meta`	`MetaStore`（MySQL `meta_rebuild`）	`backend/app/build/meta_store.py`
`ES`	`ESValueStore`（`value_index_rebuild`）	`backend/app/build/es_store.py`
`Qdrant`	`QdrantVectorStore` ×2（`column_info_rebuild` / `metric_info_rebuild`）	`backend/app/build/qdrant_store.py`

四个开关

开关	行为	是否连存储
`--dry-run`	`load_plan()` 后立刻返回，只打印 `counts()`	否
`--fresh`	`drop_database()` + 重建 schema，再全量写	是
`--verify`	只读：manifest、计数、向量维度、样例文档	只读
`--assert-idempotent`	连跑两次 `build()`，断言第二次是 `noop` 且 manifest 不变	是

四者互斥（`argparse` 的 `add_mutually_exclusive_group`，`build_meta_knowledge.py` L92–L113）。

写序（为什么不能换）

1. `load_plan()` 纯内存，不连任何存储 —— 失败最快、最便宜。
2. 非 `--fresh` 时先比对 `read_manifest()` 的 `config_hash` + 三存储计数 + 样例；全一致直接 `noop`（0 写入）。
3. 先删后建 ES / Qdrant 索引：配置变了不能残留旧值文档或旧向量。
4. **Meta 先写**（事实表），ES / Qdrant 后写（可重建索引）。
5. manifest 最后写：它记的是「这次构建成功了」这件事，所以必须最后落地（`pipeline.py` L284）。

计数（与 `facts.md` §5 一致）

目标	计数
Meta `table_info` / `column_info` / `metric_info` / `column_metric`	5 / 24 / 6 / 9 = 44 行
ES `value_index_rebuild`	76
Qdrant `column_info_rebuild`	98
Qdrant `metric_info_rebuild`	22
manifest	1
合计	**241**

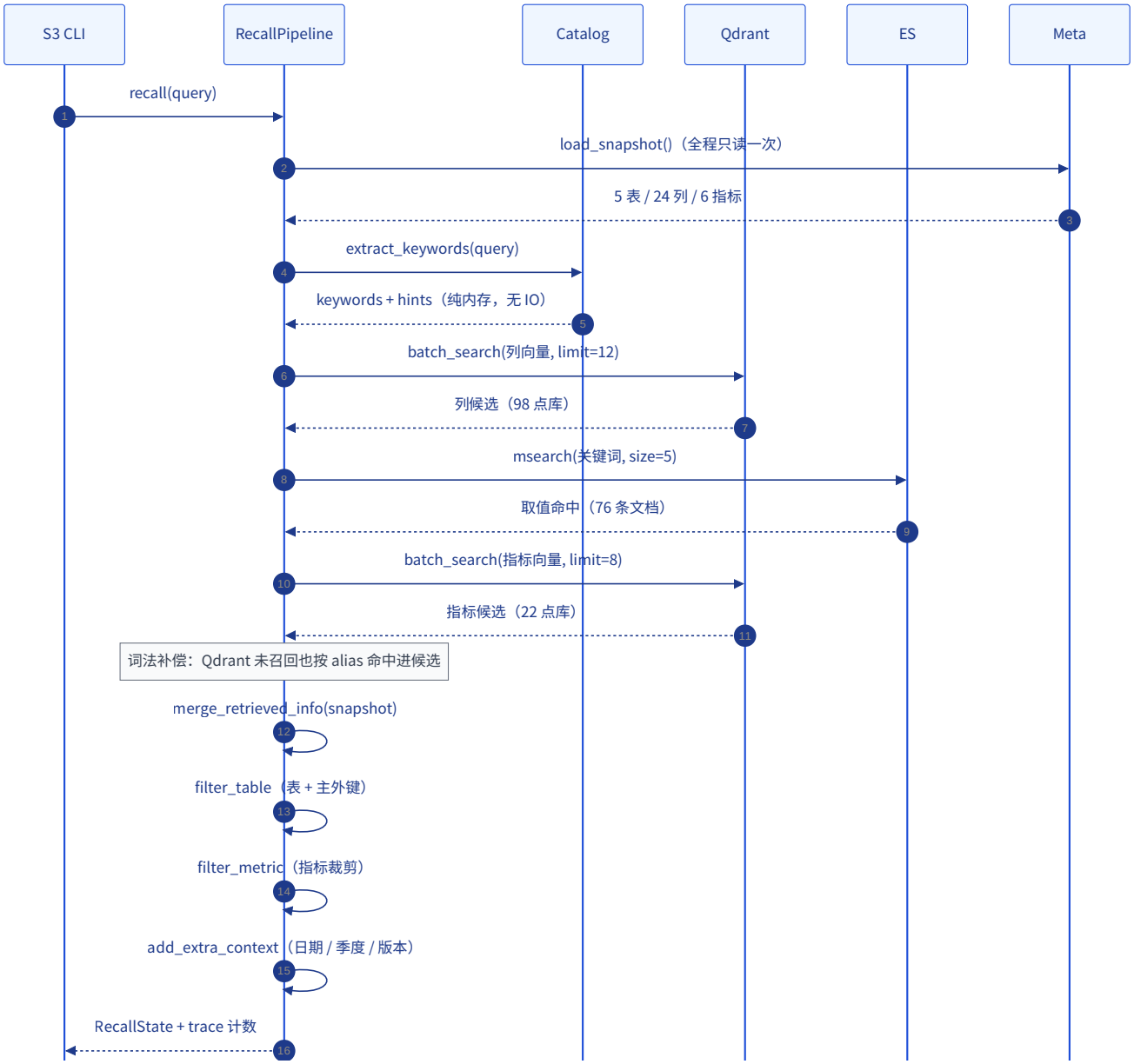
证据

结论	状态	出处
`make stage2` = dry-run → fresh → verify 三次调用	✅	`Makefile` L39–L45
计数 5/24/6/9、76、98、22	✅	`uv run --frozen python -m app.build_meta_knowledge --dry-run`（2026-10-29 本机）
	✅	`pipeline.py` L263–L284
写序 Meta → ES → Qdrant → manifest		
`noop` 判定条件（哈希 + 计数 + 健康 + 样例）	✅	`pipeline.py` L216–L236
失败也会写一条 failed manifest	✅	`pipeline.py` L174–L194、L298
写入总数 241	🟡	由 `counts()` 公式相加推导，非 `--fresh` 实测
`--fresh` 真实写三存储	⚪	本机 Docker socket 无权限

边界

- `--dry-run` 不验证存储连通性：它只能说明计划与计数，说明不了索引健康。
- `--verify` 与 `build()` 各自 `load_plan()` 一次，所以两次调用的 `config_hash` 必须一致，否则 verify 会报不一致。
- 图中 `--assert-idempotent` 与默认构建合并画在同一个分支里；区别只在它断言第二次必须 `noop`。

图 09 S3 召回流水线时序



运行时 · 读图说明见下一页

图 09 • 运行时

S3 召回流水线时序

> S3 用一次 `recall(query)` 把「关键词」变成「候选字段 / 取值 / 指标」，再按确定性规则裁剪成 `RecallState`；它**不生成 SQL、不调 LLM、不写任何存储**。

这张图回答：三存储各自被问了几次？候选怎么合并？哪些字段会被丢掉？

读图 1→2→3

1. 先看触发者（谁发起）
2. 沿编号/箭头走一遍主路径
3. 最后看失败分支与回退边

失败：看红色分支与回退边；没有回退边的步骤失败即终止。

图例：青色 = 存储；紫色 = 查询时服务。

参与者（真实进程 / 类）

图上名字	真实实体	文件
`S3 CLI`	`python -m app.scripts.recall`，用 `render()` 输出 YAML	`backend/app/scripts/recall.py`
`RecallPipeline`	`recall()`	`backend/app/recall/pipeline.py` L35
`Catalog`	`default_catalog()`，契约在内存里的投影	`backend/app/recall/catalog.py`
`Qdrant`	`stack.column_qdrant` / `stack.metric_qdrant`	`backend/app/recall/clients.py` L387
`ES`	`stack.es.msearch()`	`backend/app/recall/clients.py` L269
`Meta`	`stack.meta.load_snapshot()`	`backend/app/recall/`

图上名字	真实实体	文件
		`clients.py` L91

实际执行顺序（与直觉不同）



代码里**列召回在取值召回之前**（`pipeline.py` L73–L78），不是「先 ES 再 Qdrant」。`load_snapshot()` 也发生在最前面（L66），不是「合并时再去 Meta 取详情」——合并用的是内存快照。

三个计数（与 `facts.md` §5 一致）

库	计数	在这个流程里的角色
ES `value_index_rebuild`	76 条文档	取值召回的分母

库	计数	在这个流程里的角色
Qdrant `column_info_rebuild`	98 个点	列召回的分母（24 列拆成 98 条检索文本）
Qdrant `metric_info_rebuild`	22 个点	指标召回的分母（6 指标拆成 22 条）

过滤规则（为什么候选会变少）

- `_is_direct_column()`：`lexical_score >= 0.5` 或 `vector_score >= 0.72` 才算直接证据；`metric_dependency` 来源的字段只在 `include_dependency=True` 时保留（`nodes.py` L50–L51、L570–L580）。
- `filter_table()`：只保留「有直接证据列」的表，外加 join 路径上的主外键（`nodes.py` L582）。
- `filter_metric()`：按指标裁剪，再剪掉失去依赖的列（`nodes.py` L651、L751）。

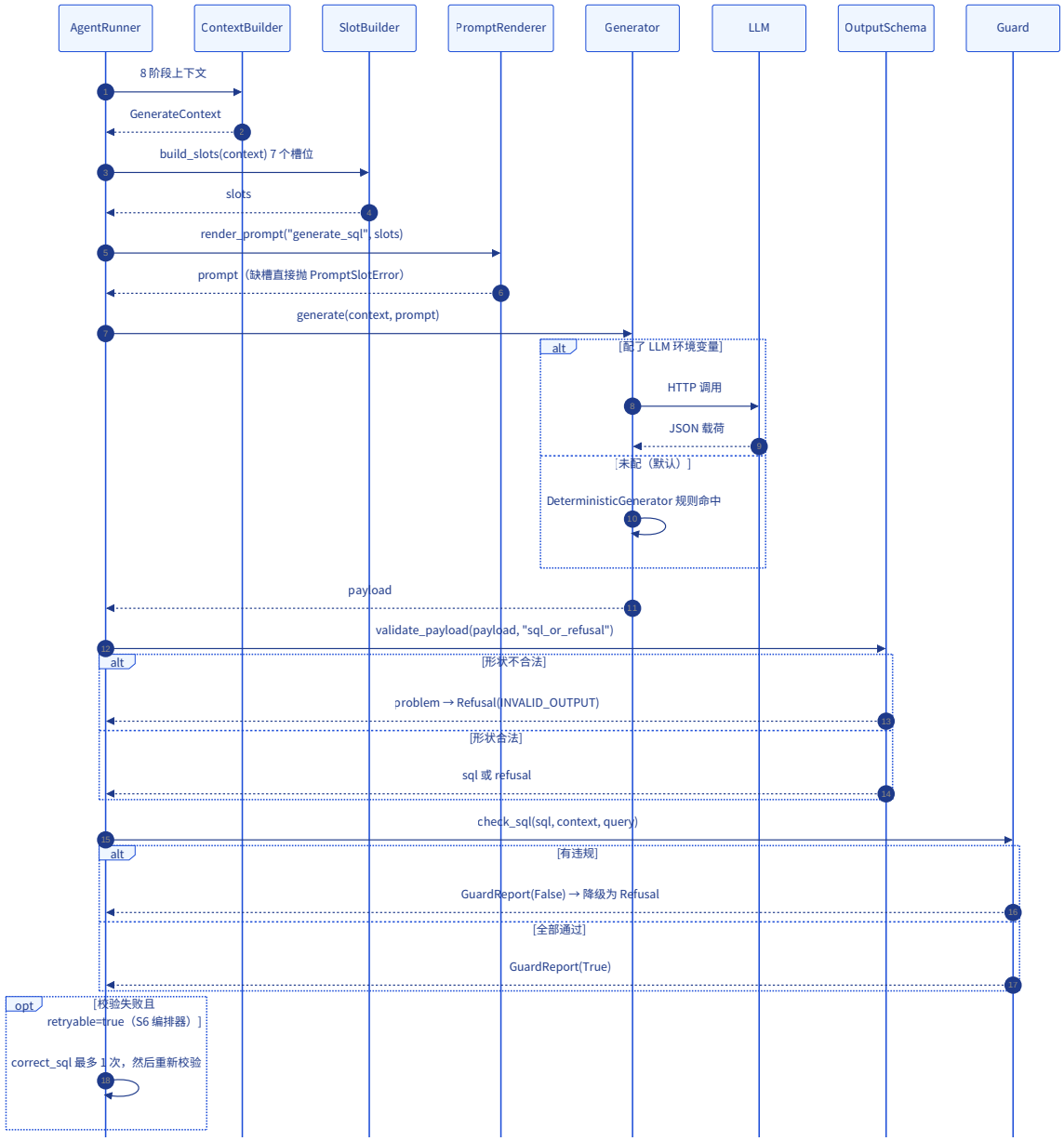
证据

结论	状态	出处
8 个节点顺序与 `steps.yaml` 一致	✅	`pipeline.py` L73–L95
三存储计数 76 / 98 / 22	✅	`--dry-run` 输出（2026-10-29 本机）
召回限额 12 / 8 / 5 / 30	✅	`nodes.py` L44–L47
词法补偿：Qdrant 未命中也能进候选	✅	`nodes.py` L164–L226（`_aggregate_candidates`）
`trace` 记录每库往返次数	✅	`pipeline.py` L95–L118
真实 Qdrant / ES 命中数（例如 76 条是否全被检索到）	⚪	需要三存储起来；本次只跑 dry-run

边界

- `recall()` 每次把四个 `roundtrips` 计数器清零（`pipeline.py` L59–L63），所以 `trace` 说的是「这道题」的往返，不是进程累计。
- `MAX_VALUE_HITS = 30` 是二次截断：先 `size=5` 取，再按分数排序截 30。
- 本图不画 SQL 生成：S3 的产物是 `RecallState`，交给 S4 的 `context_from_recall_state()`。

图 10　S4 生成 SQL 流水线



运行时 · 读图说明见下一页

图 10 • 运行时

S4 生成 SQL 流水线

> S4 把上下文压成 7 个提示词槽位交给生成器，拿到载荷后**先校验形状**（`sql_or_refusal` 契约）、**再校验内容**（sqlglot 静态规则）；两道关任一不过，产物都降级成「有名字的拒答」，不存在第三种状态。

这张图回答：上下文怎么进提示词？生成器有两种，怎么选？为什么形状校验必须在静态校验之前？

读图 1→2→3

1. 先看触发者（谁发起）
2. 沿编号/箭头走一遍主路径
3. 最后看失败分支与回退边

失败：看红色分支与回退边；没有回退边的步骤失败即终止。

图例：紫色 = 查询时服务；灰色 = 外部 LLM（未验证）；红色语义 = 违规即拒答。

参与者（真实进程 / 类）

图上名字	真实实体	文件
`AgentRunner`	`draft_sql()` / `validate_draft()` 的调用方	`backend/app/agent/orchestrator.py` L245 / L293
`ContextBuilder`	`OfflineContextBuilder` 或 `context_from_recall_state()`	`backend/app/generate/context.py` L94 / L409
`SlotBuilder`	`build_slots()`，固定 7 个槽位	`backend/app/generate/provider.py` L100
`PromptRenderer`	`render_prompt()`，缺槽抛 `PromptSlotError`	`backend/app/generate/prompts.py` L182
`Generator`	`DeterministicGenerator` / `HttpLlmGenerator`	`backend/app/generate/provider.py` L138
`LLM`	OpenAI 兼容端点（`HttpLlmGenerator.from_env()`）	`backend/app/generate/provider.py`
`OutputSchema`		
`Guard`	`validate_payload(payload, "sql_or_refusal")`	`backend/app/generate/output_schema.py` L34
`Guard`	`check_sql()`，sqlglot AST	`backend/app/generate/guard.py`

主线

1. `draft_sql()`：`build_slots()` → `render_prompt("generate_sql")` → `engine.generate()` → `validate_payload()`（`pipeline.py` L72–L133）。
2. 生成器二选一：配了 LLM 环境变量用 `HttpLlmGenerator`，否则 `DeterministicGenerator`（`main.py` L104–L110）。不配模型时 `_build_corrector()` 返回 `None`，`correct_sql` 显式 skipped。
3. `validate_draft()`：`check_sql()` 返回 `GuardReport`；`ok=False` 时取第一条违规降级为 `Refusal`（`pipeline.py` L135–L177）。

两道关的顺序为什么不能换

形状不对的产物（非 JSON、缺 `sql` / `refusal` 键）根本进不到 AST 解析——先解析会抛 `ParseError`，把「模型没按

契约说话」误报成「SQL 语法错误」。所以：`validate_payload` 在前，`check_sql` 在后。

静态校验的硬规则（`guard.py`）

规则	违规码	可重试
SQL 为空	`SQL_EMPTY`	false
多条语句	`SQL_MULTIPLE_STATEMENTS`	false
非只读（含写关键字兜底）	`SQL_NOT_READ_ONLY`	false
表 / 列不在上下文	`SQL_UNKNOWN_TABLE` / `SQL_UNKNOWN_COLUMN`	true
投影常量 / NULL	`SQL_CONSTANT_PROJECTION`	true
排名意图缺 `ORDER BY` / `LIMIT`	`SQL_MISSING_ORDER_BY` / `SQL_MISSING_LIMIT`	true
无法解析	`SQL_SYNTAX`	true

`correct_sql` 的条件性

校正**不在** S4 里发生。它由 S6 编排器在「校验失败 且 `retryable=true` 且 `attempts < max_attempts`」时插入一次（`orchestrator.py` L305–L340），`max_attempts` 来自 `contracts/errors.yaml` 的 `correction.max_attempts: 1`。没有校正器（未配 LLM）时这一步显式 skipped。

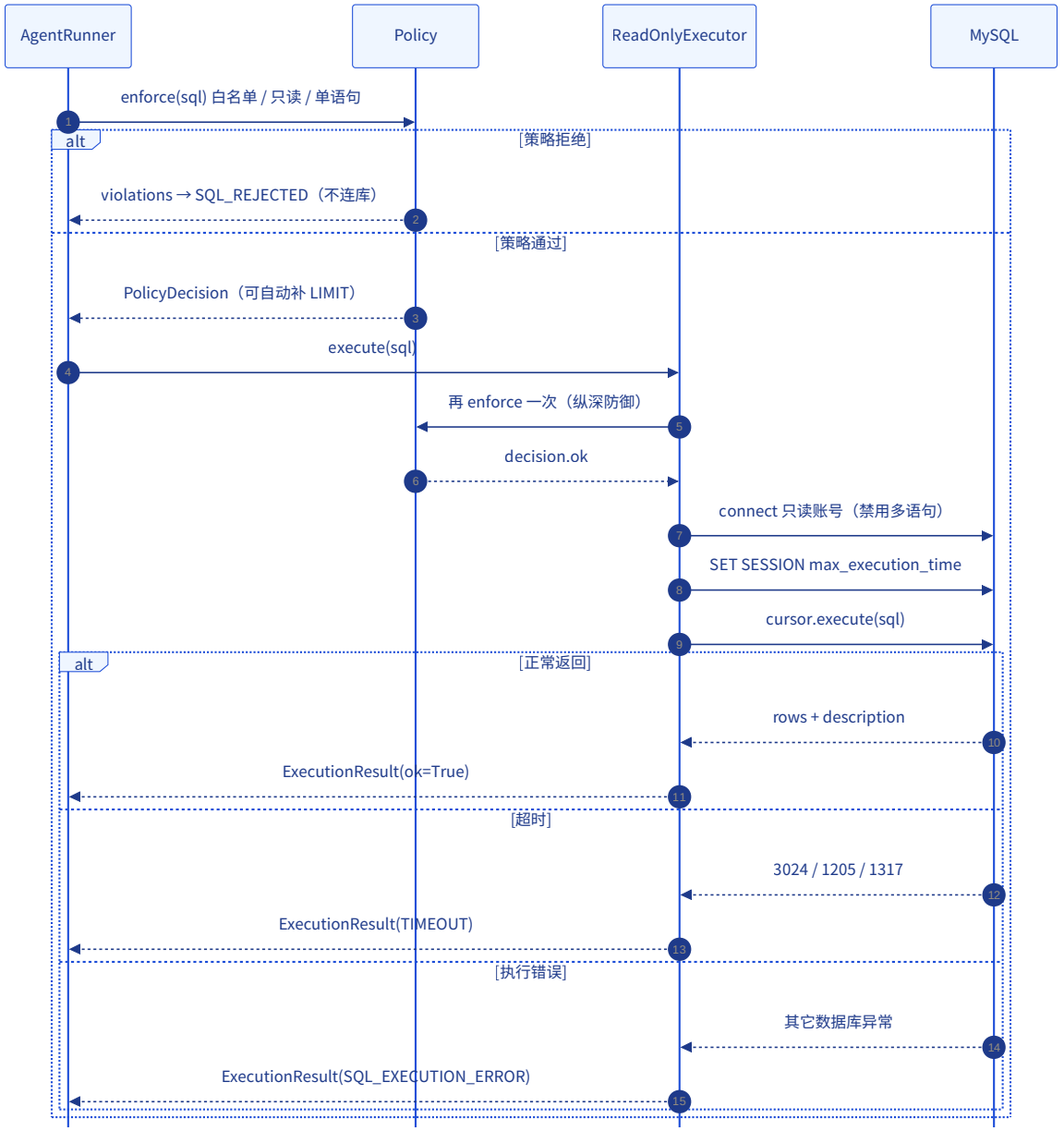
证据

结论	状态	出处
7 个槽位、缺槽即抛	✅	`provider.py` L109–L124、`prompts.py` L56
形状校验先于静态校验	✅	`pipeline.py` L112–L133 与 L156–L176
生成器默认确定性、可选 LLM	✅	`main.py` L104–L110
`max_attempts: 1`	✅	`contracts/errors.yaml` `correction` 段
真实 LLM 返回 JSON 的形状	⚪	无端点 / 无 Key，本机未跑
确定性生成器 10 题基线	🟡	需跑 `make stage4`；本次未执行

边界

- 本图不画 `correct_sql` 的完整闭环，只画它的触发条件；状态机见 12。
- `render_prompt()` 读的是 `contracts/prompts/generate_sql.prompt` 的 frontmatter，改提示词不改代码。
- `HttpLlmGenerator` 的超时 / 重试策略不在本图范围，属 S8。

图 11　S5 安全校验与执行



运行时 · 读图说明见下一页

图 11 · 运行时

S5 安全校验与执行

> 一条 SQL 先过 `policy.enforce()` 的策略判定，过了才连只读账号执行；策略拒绝、超时、执行错误是三个**互不重叠**的出口，每个都有 `contracts/errors.yaml` 里的码。

这张图回答：策略在哪两个地方各查一次？被拒绝时会不会连库？超时是怎么产生的？

读图 1→2→3

1. 先看触发者（谁发起）
2. 沿编号/箭头走一遍主路径
3. 最后看失败分支与回退边

失败：看红色分支与回退边；没有回退边的步骤失败即终止。

图例：红色 = 安全与失败；青色 = MySQL。

参与者（真实进程 / 类）

图上名字	真实实体	文件
`AgentRunner`	`_execute()` → `executor.execute()`	`backend/app/agent/orchestrator.py` L444
`Policy`	`enforce(sql, whitelist, config)`	`backend/app/security/policy.py` L298
`ReadOnlyExecutor`	`execute()` / `_run()`	`backend/app/security/executor.py` L226 / L243
`MySQL`	`dw_reader` 只读账号	`backend/app/security/executor.py` L187

三个出口

出口	触发	错误码	是否连库
策略拒绝	多语句 / 写操作 / 非白名单表 / 缺 `ORDER BY` · `LIMIT` / 可执行注释	`SQL_REJECTED`	**否**（`execute()` L229–L241 直接返回）
超时	MySQL `3024` / `1205` / `1317`，或客户端 `2006` / `2013`	`TIMEOUT`	是
执行错误	其它数据库异常	`SQL_EXECUTION_ERROR`	是

错误码映射在 `_map_error()`（`executor.py` L136–L159）。

两次策略检查（纵深防御）

1. **校验步骤**：`validate_sql(gates=("guard","policy"))` 会先跑 `enforce()`，让「这条 SQL 不该被执行」在执行**之前**就变成一条错误事件（`backend/app/agent/validate.py` L56–L70）。
2. **执行器内部**：`execute()` 再查一次（`executor.py` L228）。别的调用方（`secure_execute`、测试、脚本）绕不过这一层。

连接层的三道保险

- 只读账号：`user = config.account_user`（`dw_reader`），密码只从 `REBUILD_DW_RO_PASSWORD` 读，缺了直接

`ExecutorError`，**不降级**（`executor.py` L32、L187–L196）。

- 禁用多语句：`CLIENT.CAPABILITIES & ~CLIENT.MULTI_STATEMENTS`（L204）。
- 服务端超时：`SET SESSION max_execution_time = ...`，设置失败会抛（`_prepare_session()` L218–L223）。

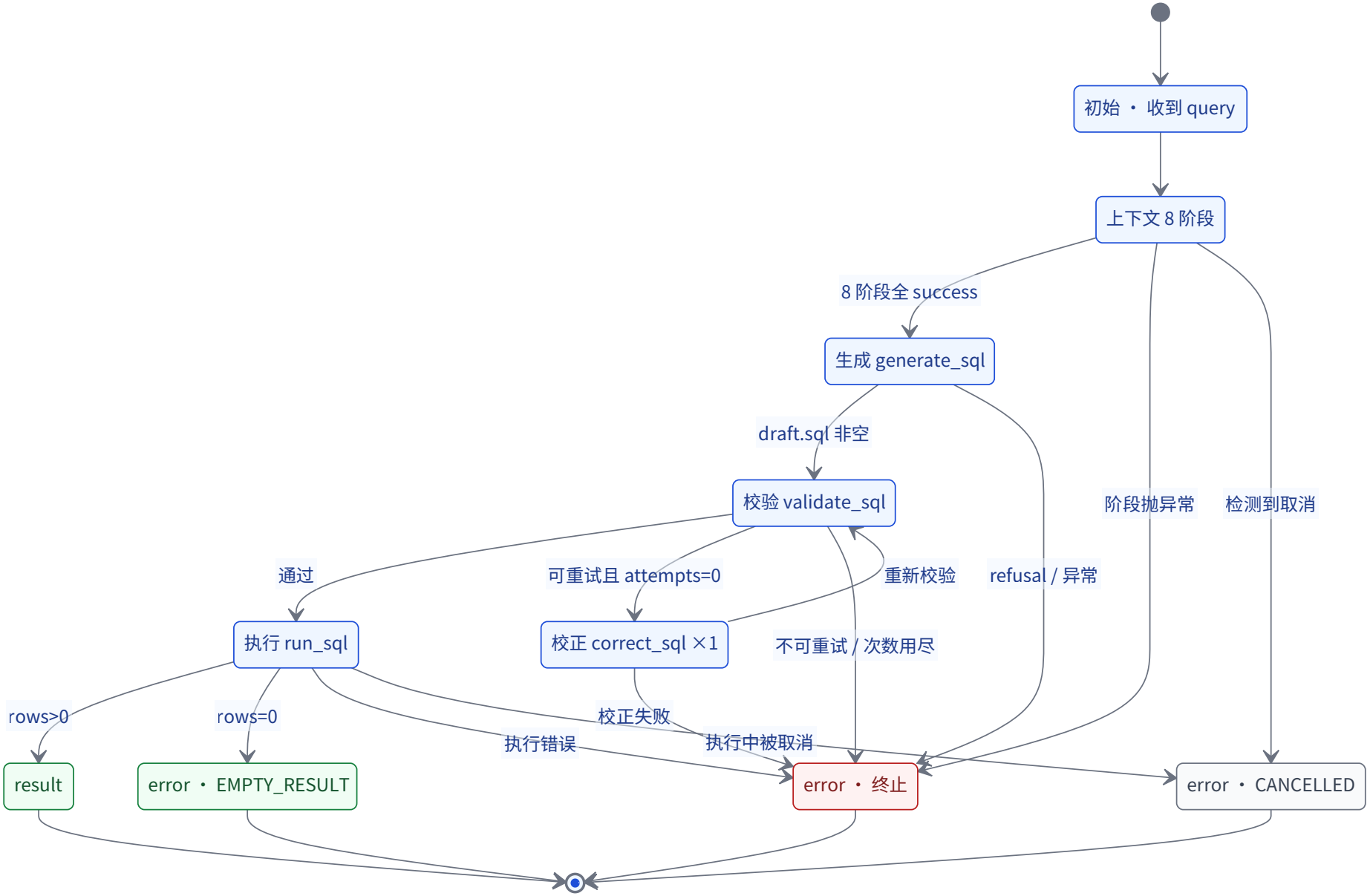
证据

结论	状态	出处
策略不通过时**不连库**	✅	`executor.py` L226–L241
两次 `enforce()`	✅	`validate.py` L56–L70 + `executor.py` L228
`max_execution_time` 会话级设置	✅	`executor.py` L218–L223
错误码 3024/1205/1317/2006/2013 映射	✅	`executor.py` L65–L71、L136–L159
真实 MySQL 超时行为	⚪	需要三存储起来；本次未连库
`contracts/sql.policy.yaml` 的具体规则集	🟡	本图只画判定位置，不逐条列举

边界

- 执行阶段的失败**不再校正**：能走到这里说明 SQL 已过 guard + policy，数据库层的错误不是「最小必要修正」能可靠修好的（`orchestrator.py` L392–L415 注释）。
- `_prepare_session()` 失败会抛 `ExecutorError`，被 `_run()` 重新抛出，最终由编排器兜成 `INTERNAL_ERROR` ——**不会**被当成「这条 SQL 失败了」。
- 本图不含结果集转换（`to_result_payload()`），那在 07 与 13。

图 12　S6 校正闭环状态机



运行时 · 读图说明见下一页

图 12 · 运行时

S6 校正闭环状态机

> 一次问数是一条**单向前进、只允许一次回退**的状态机：校验失败且 `retryable=true` 时回到「校验」重来一次；`retryable=false` 或次数用尽就直接终止，执行阶段的失败永不回退。

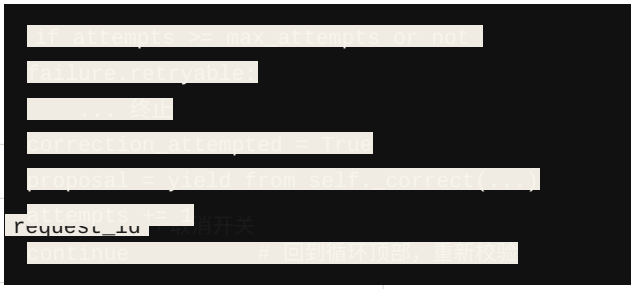
这张图回答：什么情况下会走 `correct_sql`？走完之后为什么必须重新校验？哪些失败是不可重试的？

读图 1→2→3

1. 先看触发者（谁发起）
2. 沿编号/箭头走一遍主路径
3. 最后看失败分支与回退边

失败：看红色分支与回退边；没有回退边的步骤失败即终止。

图例：绿色 = 正常收尾；红色 = 失败收尾；灰色 = 取消收尾。



状态与真实事件

状态	对应 `steps.yaml` 步骤	真实工作
初始 · 收到 query	—	`query()` 建 `request_id`
上下文 8 阶段	前 8 个 step	`OfflineContextBuilder.phases()`
生成 generate_sql	`generate_sql`	`draft_sql()`
校验 validate_sql	`validate_sql`	`validate_sql(gat`
校正 correct_sql ×1	`correct_sql`（`conditional: true`）	`propose_correcti`
执行 run_sql	`run_sql`	`ReadOnlyExecutor.execute()`
result	—	`to_result_payload()`
error · EMPTY_RESULT	`run_sql` 仍为 success	0 行，`class=empty`
error · 终止	出错的那一步	码来自 `errors.yaml`
error · CANCELLED	—	`class=cancelled`

唯一的回退边

`校验 → 校正 → 校验` 是全图**唯一**的环，条件写死在 `orchestrator.py` L305：

- `max_attempts` 来自 `contracts/errors.yaml` 的 `correction.max_attempts: 1`。
- `retryable` 来自错误码表：`SQL_REJECTED` / `SQL_NOT_READ_ONLY` / `SQL_MULTIPLE_STATEMENTS` / `SQL_EMPTY` / `EMPTY_RESULT` / `TIMEOUT` / `CANCELLED` / `LLM_REFUSAL` 都是 `false`。
- 重试必须**重新过 guard + policy**（`revalidate: true`，`revalidate_with: [guard, policy]`）——校正后的 SQL 不享受任何豁免。

`retryable=false` 为什么直接终止

安全策略的拒绝永远不可重试：把被拒的 SQL 再喂给模型「改一下」，等于给注入开了一条重试通道（`contracts/errors.yaml` 顶部注释）。

证据

结论	状态	出处
只有一条回退边，且最多 1 次	✅	`orchestrator.py` L282、L305–L340

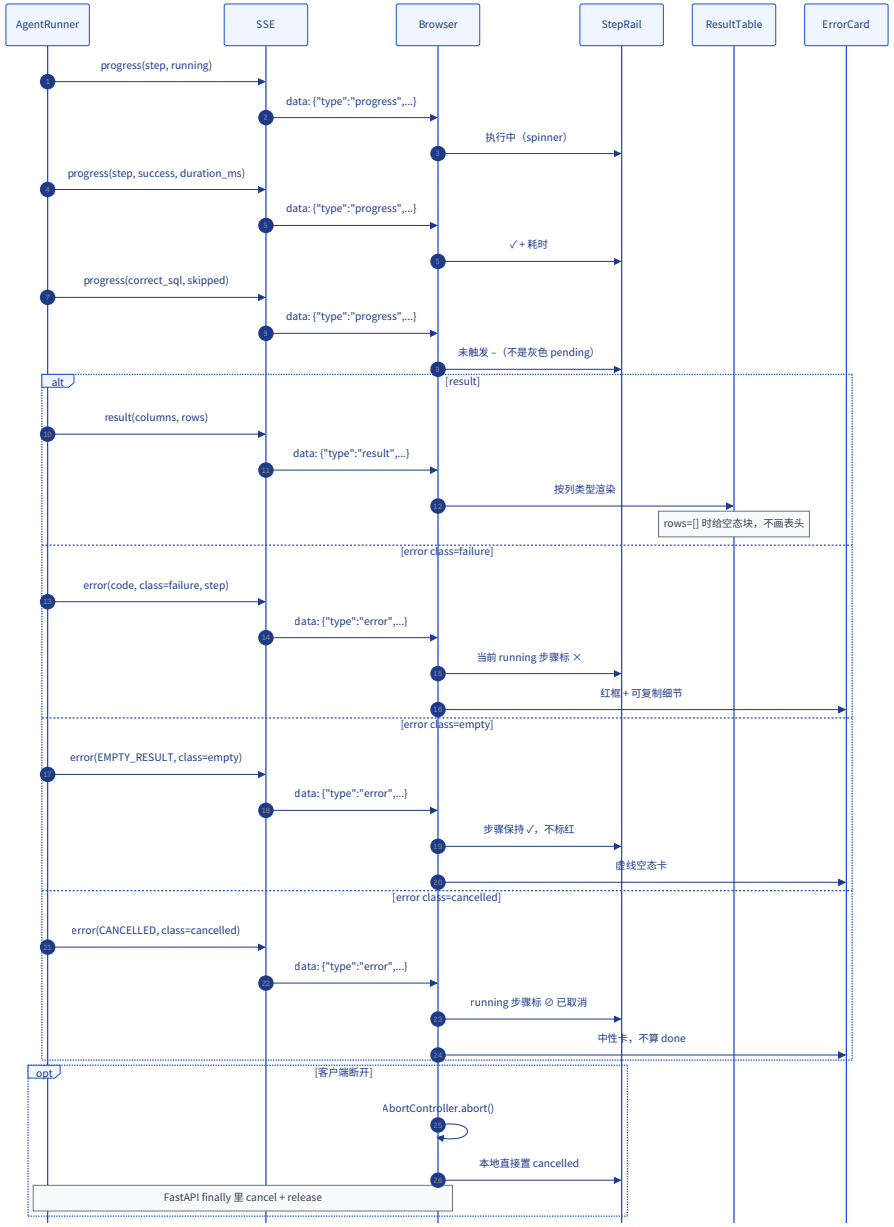
结论	状态	出处
`retryable` 从错误码表读	✅	`backend/app/agent/errors.py` L292
`max_attempts: 1` + `revalidate`	✅	`contracts/errors.yaml` `correction` 段
执行阶段失败不校正	✅	`orchestrator.py` L392–L415
每个状态在 SSE 上都有 running/success/error/skipped	✅	`orchestrator.py` L168–L180、L426–L440

结论	状态	出处
一次真实校正的往返	⚪	需要 LLM；本次未跑

边界

- 状态名是中文标签 + ASCII id，便于在 Mermaid 里稳定渲染；id 与代码无耦合。
- 「校正失败」包含两种情况：没有校正器（`corrector is None`）与校正器返回 `ok=False`。
- `EMPTY_RESULT` 是**收尾状态**，不是失败：`run_sql` 那一步在 SSE 上仍是 `success`。

图 13　S7 SSE 事件序列



运行时 · 读图说明见下一页

图 13 • 运行时

S7 SSE 事件序列

> 后端只发三种事件（`progress` / `result` / `error`），前端按 `type` 与 `class` 分流到三个组件；`skipped` 和 `cancelled` 在界面上各占一个独立状态，不并入 pending 或失败。

这张图回答：一条 `progress` 到哪个组件？`error.class` 三个取值各长什么样？取消为什么不算 done?

读图 1→2→3

1. 先看触发者（谁发起）
2. 沿编号/箭头走一遍主路径
3. 最后看失败分支与回退边

失败：看红色分支与回退边；没有回退边的步骤失败即终止。

图例：绿色语义 = 成功路径；红色 = failure；灰色 = cancelled。

参与者（真实进程 / 类）

图上名字	真实实体	文件
`AgentRunner`	事件生产者	`backend/app/agent/orchestrator.py` L199
`SSE`	`_sse()` 帧编码	`backend/app/main.py` L250
`Browser`	`App` 的 `switch (event.type)`	`frontend/src/App.tsx` L141–L176
`StepRail`	`StepRail` 组件	`frontend/src/components/StepRail.tsx` L33
`ResultTable`	`ResultTable` 组件	`frontend/src/components/ResultTable.tsx` L46
`ErrorCard`	`ErrorCard` 组件	`frontend/src/components/ErrorCard.tsx` L32

三种事件 → 三个组件

事件	判定	UI
`progress` `running`	`sse.schema.json` `status` 枚举	步骤条 spinner
`progress` `success`	同上，`duration_ms` 必须是数字	✓ + 耗时
`progress` `skipped`	条件步骤未触发（`orchestrator.py` L168–L180）	「未触发 –」
`result`	`{columns:[{name,type}], rows:[[...]]}`	`ResultTable`，按列类型渲染
`error` `class=failure`	红框 + 失败步骤打 ×	`ErrorCard`
`error` `class=empty`	空态卡，**步骤不标红**	`ErrorCard`（虚线）
`error` `class=cancelled`	取消态，running 步骤标 ⊘	`ErrorCard`（中性）

`class` 只有三个合法取值（`contracts/sse.schema.json` `error.class` 枚举 = `failure` / `empty` / `cancelled`），由 `contracts/errors.yaml` 的 `class` 字段决定。

`skipped` 的 UI 语义

条件步骤 `correct_sql` 要么跑（running → success/error），要么**显式** skipped。后端在三条提前收尾路径上补发这一条（`orchestrator.py` L168–L180 的 `_skip_correct()`），前端 `StepRail` 渲染成 `–` + 「未触发」（`StepRail.tsx` L51–L53）。**不允许**永远是一块灰色 pending——那正是审计缺陷 D4。

`cancelled` 的 UI 语义

- 后端：发 `error` + `CANCELLED`，`class=cancelled`。
- 前端：`event.class === "cancelled"` → `setStatus("cancelled")`，并把所有 running 步骤改成 `cancelled`（`App.tsx` L171–L175）。
- 本地 abort：`isAbortError` 分支同样置 `cancelled`（L210–L217），**不是** done。旧系统 `App.tsx:149` 把 abort 写成 `status: "done"`，那就是被修掉的缺陷。

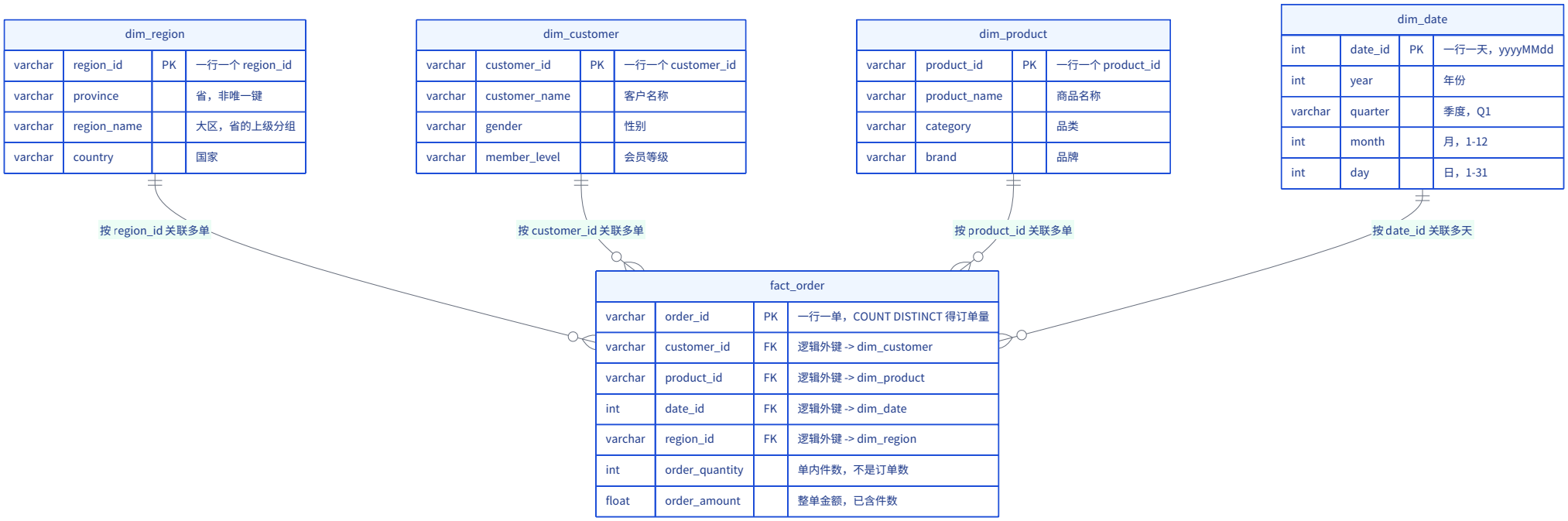
证据

结论	状态	出处
只有 3 种事件、`progress.status` 只有 4 值	✅	`contracts/sse.schema.json` `$defs`
`error.class` 三值决定 UI	✅	`ErrorCard.tsx` L13–L17、L52
`skipped` 由后端显式补发	✅	`orchestrator.py` L168–L180
取消 ≠ done	✅	`App.tsx` L171–L175、L210–L217
未知事件类型只记录、不当失败	✅	`App.tsx` L187–L194、`api.ts` L129
真实浏览器渲染	⚪	本次未起前端

边界

- **`ResultTable` 的 `rows.length === 0` 分支在生产路径不可达**：后端在 0 行时发的是 `EMPTY_RESULT`，不是空 `result`（`orchestrator.py` L373–L388）。那段代码是给其它调用方与契约测试用的兜底。
- 前端对未知 `step` 会抛 `UNKNOWN_STEP`（`App.tsx` L141–L144）——步骤 id 必须来自 `/api/steps`。
- 流「静默结束」（没收到 result/error 就断了）被前端判成 `STREAM_INCOMPLETE` 失败（L199–L209），不允许当成功。

图 14　数仓 ER 图（5 表）



数据 · 读图说明见下一页

图 14 • 数据

数仓 ER 图（5 表）

> 数仓只有 5 张表；4 张维度表各由主键唯一确定一行，`fact_order` 一行一单，四个 `*_id` 是指向维度的逻辑外键。

这张图回答：每张表一行代表什么？主键是什么？事实表如何关联维度？哪些列是真实存在、哪些是常见误问（`city`、`price`）？

读图 1→2→3

1. 先看实体与 grain（一行代表什么）
2. 再看主键/外键与关系
3. 最后看约束、值域与口径

失败：越界时间、缺列、错 grain 会在校验阶段被拒。

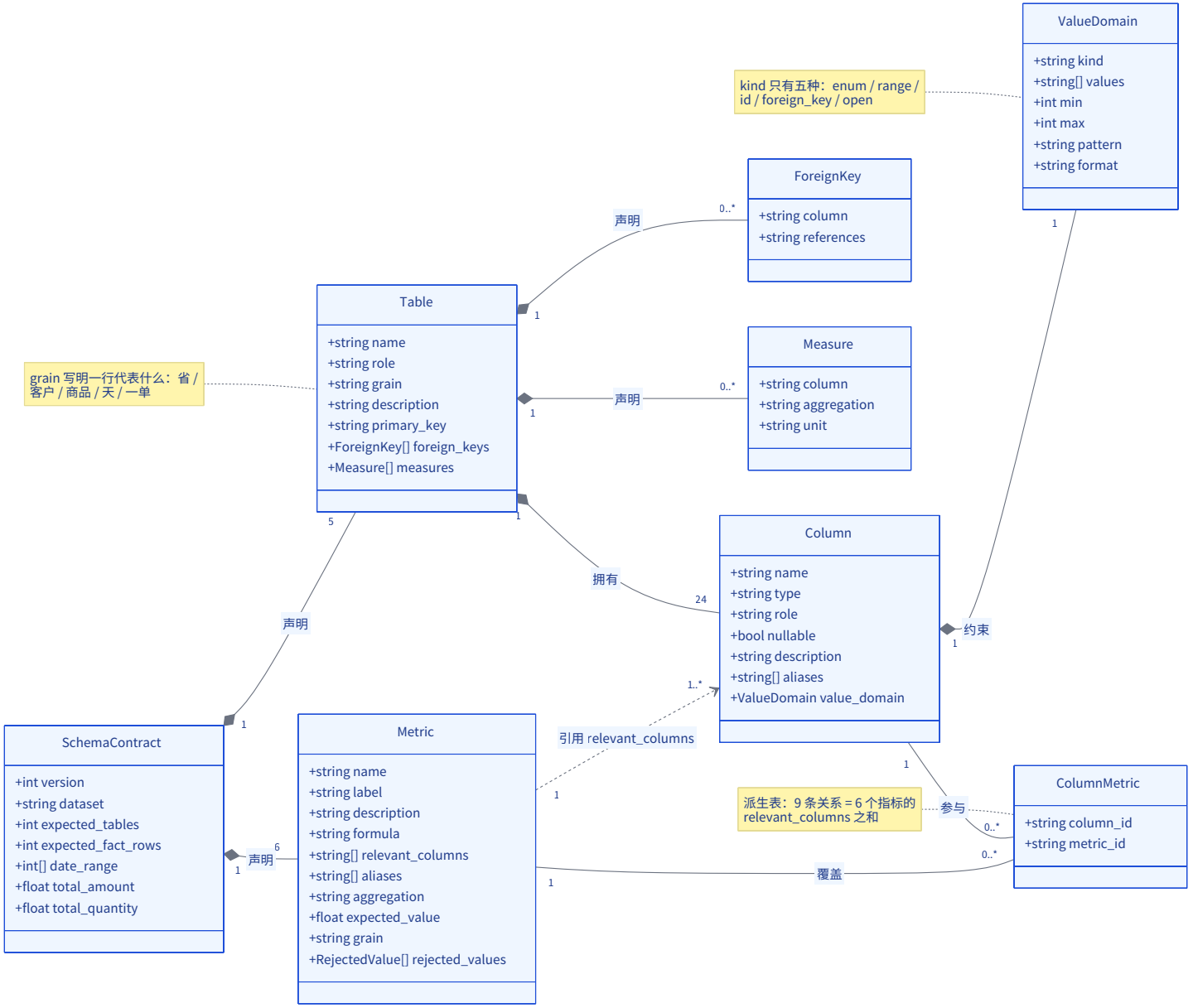
图例：蓝色=契约与事实源；`PK`=主键，`FK`=逻辑外键；属性后的引号是粒度说明，不是物理约束。

关键元素：- `dim_region`：`region_id` 主键；`region_name` 是大区（省的上级分组），**不是唯一键**；本表没有 `city` 列。- `dim_customer`：`customer_id` 主键。- `dim_product`：`product_id` 主键；本表没有 `price` 列，金额只在 `fact_order.order_amount`。- `dim_date`：`date_id` 主键，一行一天，`yyyyMMdd`。- `fact_order`：`order_id` 主键，**一行一单**；`order_quantity` 是单内件数，`order_amount` 是整单金额。- 四条逻辑外键：`customer_id`、`product_id`、`date_id`、`region_id`。

证据：- `contracts/schema.contract.yaml` L26–L399：5 表、主键、`grain`、`foreign_keys`。- `fixtures/dw.sql` L13–L323：物理 DDL 与 115 行订单数据。- `design/inventory/facts.md` L69–L88：表粒度与外键清单。

边界：外键是**逻辑**关系，DDL 中没有 `FOREIGN KEY` 约束；图中不声称数据库会强制引用完整性。列类型在图中简写为 `varchar/int/float`，精确类型（如 `VARCHAR(20)`、`FLOAT`）以契约与 DDL 为准。

图 15　契约模型（表 / 列 / 指标 / 值域 / 列—指标）



数据 · 读图说明见下一页

图 15 • 数据

契约模型（表 / 列 / 指标 / 值域 / 列—指标）

> `schema.contract.yaml` 把「表、列、指标、值域、列—指标关系」建模成五个对象；列—指标关系共 9 条，是 6 个指标 `relevant_columns` 的并集。

这张图回答：契约里到底有哪些对象？每个对象带哪些字段？`ValueDomain` 的 `kind` 有哪几种？`ColumnMetric` 为什么是派生表？

读图 1→2→3

1. 先看实体与 grain（一行代表什么）
2. 再看主键/外键与关系
3. 最后看约束、值域与口径

失败：越界时间、缺列、错 grain 会在校验阶段被拒。

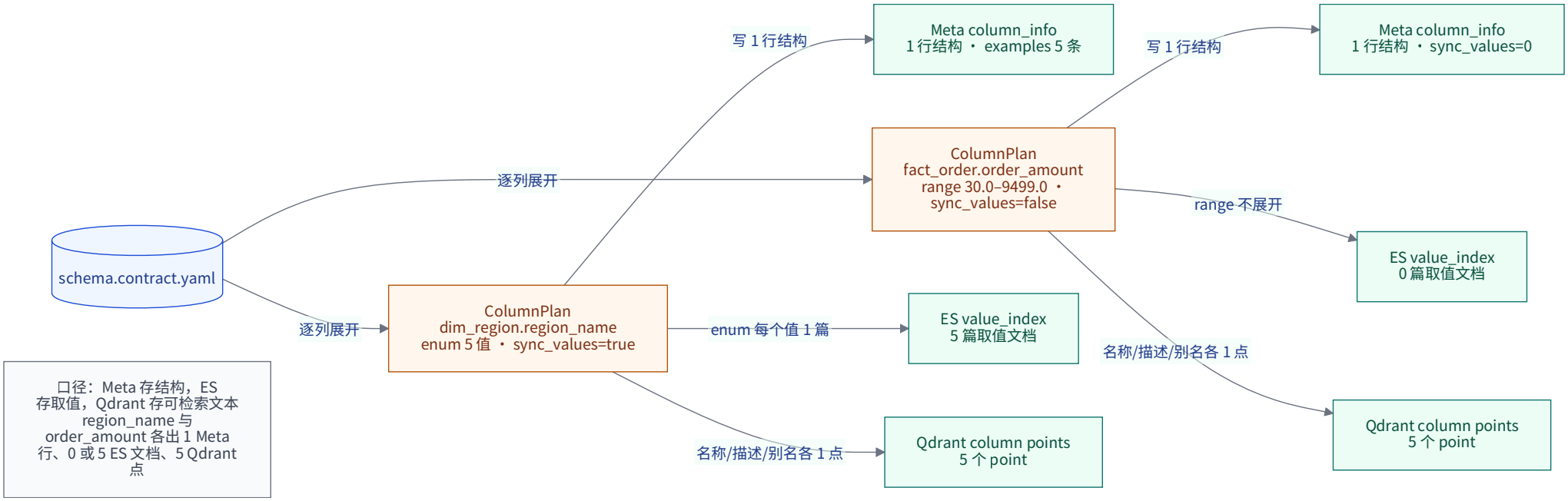
图例：蓝色=契约对象；菱形箭头=引用；`*--` =组合；黄色便签=口径说明。

关键元素：- `SchemaContract`：顶层 `version`、`dataset`、`tables`、`metrics`。- `Table`：`name`、`role`（fact/dim）、`grain`、`primary_key`、`columns`、`foreign_keys`、`measures`。- `Column`：`name`、`type`、`role`、`nullable`、`aliases`、`value_domain`。- `Metric`：`name`、`label`、`formula`、`aggregation`、`expected_value`、`relevant_columns`、`rejected_values`。- `ValueDomain.kind`：`enum` / `range` / `id` / `foreign_key` / `open` 五种。- `ColumnMetric`：`column_id` + `metric_id`；契约里不单独写，由 6 个指标的 `relevant_columns` 展开得到 9 条。

证据：- `contracts/schema.contract.yaml` L15–L399：所有字段与 5 表 / 24 列 / 6 指标。- `backend/app/build/plan.py` L69–L192：`TablePlan` / `ColumnPlan` / `MetricPlan` / `BuildPlan.column_metric_pairs()`。- `backend/app/build/meta_store.py` L66–L70：`column_metric` 复合主键 `(column_id, metric_id)`。- `backend/app/build_meta_knowledge.py --dry-run`（✅2026-10-27 本机，计数 5/24/6/9）。

边界：`ForeignKey` 与 `Measure` 是契约内的匿名子结构，不是独立数据库表；图中把它们画成类是为了看清字段来源。`ColumnMetric` 是 `BuildPlan` 的派生结果，不来自 YAML 里的独立节点。

图 16　一份契约如何落到三存储（region_name / order_amount）



数据 · 读图说明见下一页

图 16 • 数据

一份契约如何落到三存储（region_name / order_amount）

> 同一份契约列，Meta 得到 1 行结构描述，ES 按值域决定是否展开取值文档，Qdrant 把名称、描述、每个别名各变成一个可检索 point。

这张图回答：`region_name` 和 `order_amount` 分别变成什么？为什么一个进 ES、一个不进？Qdrant 的 98 个 point 是怎么来的？Meta 的 `examples` 能不能当全集？

读图 1→2→3

1. 先看实体与 grain（一行代表什么）
2. 再看主键/外键与关系
3. 最后看约束、值域与口径

失败：越界时间、缺列、错 grain 会在校验阶段被拒。

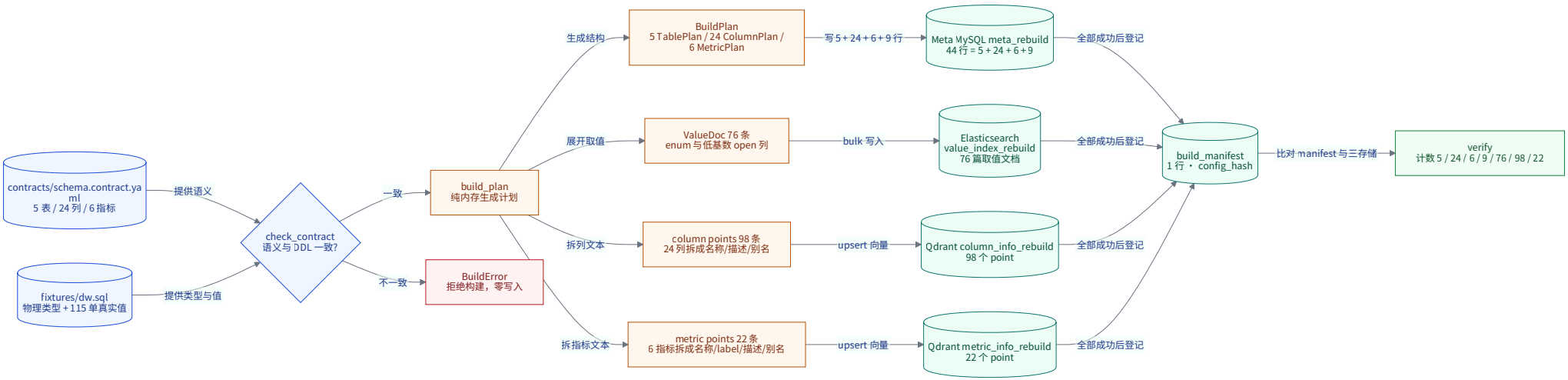
图例：蓝色=契约；橙色=BuildPlan；青色=三存储；灰色=口径说明。

关键元素：- `dim_region.region_name`：`kind=enum`，5 个值（华南/华东/西南/华北/华中），`role=dimension`，`sync_values=true`。- Meta：`column_info` 1 行，`examples` 只取排序后前 10 条（本列共 5 条）。- ES：5 篇文档，`id=dim_region.region_name.华南` 这类，`value` + `column_id`。- Qdrant：`region_name`、描述、`地区`、`区域`、`大区` 共 5 个 point。- `fact_order.order_amount`：`kind=range`（30.0–9499.0），`role=measure`，`sync_values=false`。- Meta：`column_info` 1 行，`value_domain` 原样存 JSON，`sync_values=0`。- ES：0 篇文档；range 不进入取值索引。- Qdrant：`order_amount`、描述、`订单金额`、`收入`、`成交金额` 共 5 个 point。- 24 列合计 98 个 column point；6 指标另有 22 个 metric point。

证据：- `contracts/schema.contract.yaml` L53–L61（`region_name`）、L288–L309（`order_amount`）。- `backend/app/build/plan.py` L209–L225：`_should_sync` 只放行 dimension + enum/open + 基数阈值。- `backend/app/build/plan.py` L235–L243：点文本 = 名称 + 描述 + 别名，去重。- `backend/app/build/plan.py` L34：`EXAMPLES_LIMIT = 10`，Meta 的 `examples` 是显式截断。- `backend/app/build/meta_store.py` L36–L48：`column_info` 字段；L254–L272 写入。- `backend/app/build/es_store.py` L21–L33：`value` / `column_id` / `id` 三个字段。- `backend/app/build/qdrant_store.py` L113–L145：按 batch upsert point 与向量。- `backend/app/build_meta_knowledge.py --dry-run`（✅ 2026-10-27）：76 / 98 / 22。

边界：图中箭头表示「由 BuildPlan 写往存储」的构建方向；查询时三个存储之间没有数据流动。`region_name` 的 5 篇 ES 文档是按本数据集 6 行 `dim_region` 去重后的结果；换数据集计数会变，但规则不变。

图 17　S2 数据 lineage（契约 + 冻结数仓 → 三存储 + manifest）



数据 · 读图说明见下一页

图 17 • 数据

S2 数据 lineage（契约 + 冻结数仓 → 三存储 + manifest）

> S2 先把契约与 DDL 校验成一份纯内存 `BuildPlan`，再按 Meta → ES → Qdrant → manifest 的顺序写入；任何一步失败都不会登记成功的 manifest。

这张图回答：数据从哪来？在哪一步校验？三存储各写多少？`5/24/6/9/76/98/22` 分别是什么？manifest 为什么是构建成功的唯一凭据？

读图 1→2→3

1. 先看实体与 grain（一行代表什么）
2. 再看主键/外键与关系
3. 最后看约束、值域与口径

失败：越界时间、缺列、错 grain 会在校验阶段被拒。

图例：蓝色=输入与契约校验；橙色=BuildPlan 生成；青色=存储；绿色=验收；红色=拒绝构建。

关键元素：- 输入：`contracts/schema.contract.yaml`（语义）+ `fixtures/dw.sql`（物理类型与真实值）。- `check_contract`：语义与 DDL 不一致时抛 `BuildError`，零写入。- `build_plan`：生成 5 个 `TablePlan`、24 个 `ColumnPlan`、6 个 `MetricPlan`、76 个 `ValueDoc`、98 个 column point、22 个 metric point。- Meta：44 行 = `table_info` 5 + `column_info` 24 + `metric_info` 6 + `column_metric` 9。- ES：76 篇取值文档；Qdrant：98 + 22 个 point。- `build_manifest`：1 行，记录 `config_hash`、目标名、7 个计数与 `status`。- 推导写入总数：44 + 76 + 98 + 22 + 1 = **241**。

证据：- `backend/app/build/plan.py` L281–L400：`build_plan` 与重复 ID 检查。- `backend/app/build/plan.py` L174–L185：`counts()` 的 7 个键。- `backend/app/build/pipeline.py` L196–L291：构建顺序、`meta.reset_data` → `write_plan` → `bulk_index` → 两个 `upsert` → `write_manifest`。- `backend/app/build/meta_store.py` L25–L95：5 张 Meta 表 DDL。- `backend/app/build/es_store.py` L107–L141：bulk 写入 `value_index_rebuild`。- `backend/app/build/qdrant_store.py` L113–L145：`column_info_rebuild` / `metric_info_rebuild` 写入。- `backend/app/build_meta_knowledge.py --dry-run`（✅2026-10-27 本机）：5/24/6/9/76/98/22。- `design/inventory/facts.md` L52–L67：计数与 241 的推导。

边界：241 是按代码公式推导，不是 `--fresh` 实测（本机 Docker socket 无权限），标 🟡。manifest 的 `status=failed` 也会落一行，用于记录失败；只有 `status=success` 且哈希、计数、维度、样例都一致时 `verify` 才通过。

图 18 Meta 表结构（5 张元数据表）



数据 · 读图说明见下一页

图 18 • 数据

Meta 表结构（5 张元数据表）

> Meta 用 5 张表描述「有哪些表、哪些列、什么指标、列和指标怎么关联、这次构建是否成功」；前 4 张是知识，`build_manifest` 是构建凭据。

这张图回答：每张 Meta 表的字段和主键是什么？`column_info` 怎么指向表？`column_metric` 的复合主键是什么？manifest 记录哪些计数？

读图 1→2→3

1. 先看实体与 grain（一行代表什么）
2. 再看主键/外键与关系
3. 最后看约束、值域与口径

失败：越界时间、缺列、错 grain 会在校验阶段被拒。

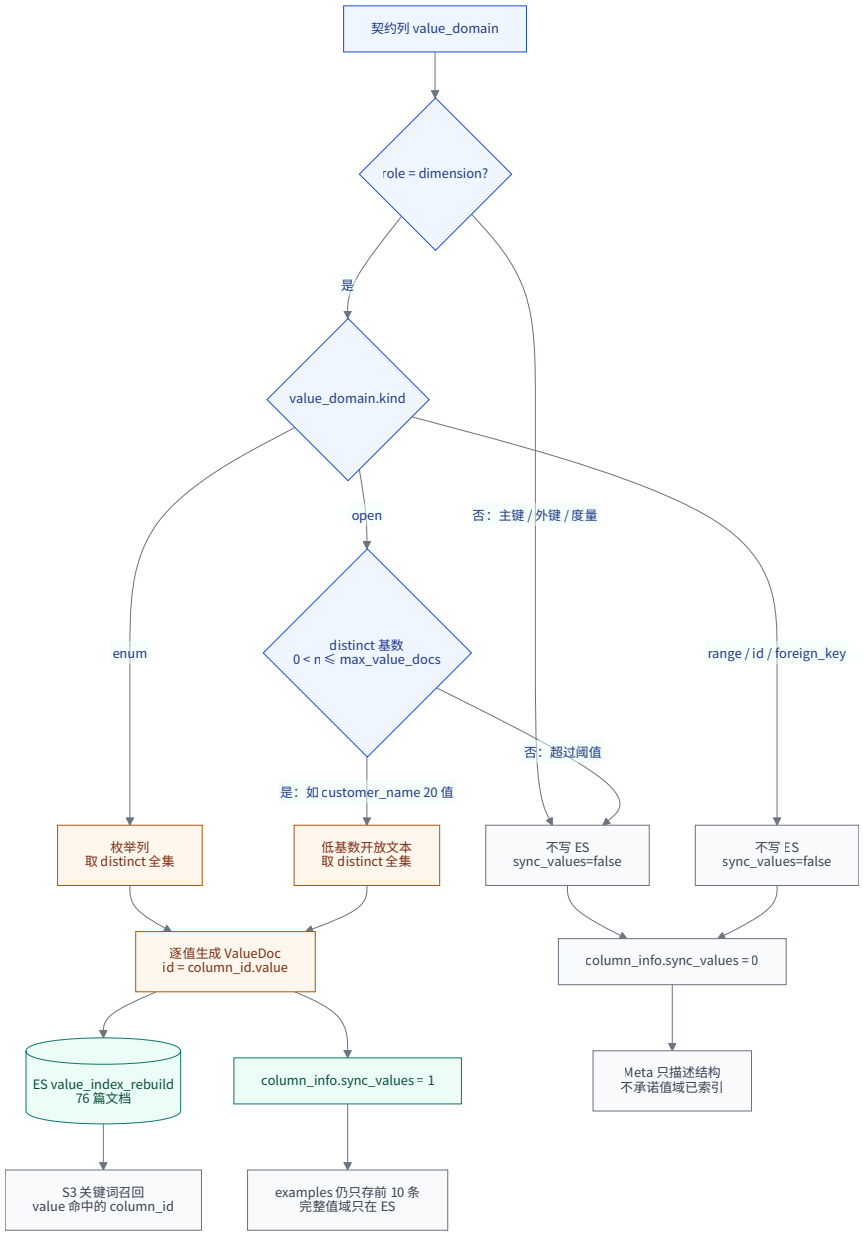
图例：蓝色=Meta 表；`PK` =主键，`UK` =唯一键，`FK` =逻辑外键（DDL 未声明物理外键）。

关键元素：- `table_info`：`id` 主键（表名），`name`、`role`、`description`、`grain`。- `column_info`：`id` 主键（`table.column`），`name`、`type`、`role`、`examples` (JSON)、`description`、`alias` (JSON)、`table_id`、`value_domain` (JSON)、`sync_values`；`table_id` 建了索引。- `metric_info`：`id` 主键（指标名），`name`、`label`、`description`、`formula`、`relevant_columns` (JSON)、`alias` (JSON)、`aggregation`、`expected_value`、`grain`、`rejected_values` (JSON)。- `column_metric`：复合主键 `(column_id, metric_id)`，两列分别逻辑指向 `column_info.id` 与 `metric_info.id`。- `build_manifest`：自增 `seq` 主键；`build_id` 唯一；记录 `manifest_version`、`built_at`、`config_hash`、`contract_version`、嵌入模型/维度、7 个计数、`es_analyzer`、`targets_json`、`status`、`error`。

证据：- `backend/app/build/meta_store.py` L25–L95：建表语句与主键/索引。- `backend/app/build/meta_store.py` L236–L304：`write_plan` 写入 4 张知识表。- `backend/app/build/meta_store.py` L306–L341：`write_manifest`。- `backend/app/build/meta_store.py` L347–L368：`read_manifest` 取最新一行并把 `targets_json` 还原成 `targets`。- `backend/app/build/plan.py` L174–L192：7 个计数键与 `column_metric_pairs()`。

边界：`column_info.table_id`、`column_metric.*` 都只是**逻辑外键**，DDL 中没有 `FOREIGN KEY`。`examples` 是排序后前 10 条，不是完整值域；完整枚举值在 ES。`build_manifest.status` 允许 `success` / `failed`，失败也会留一行用于排查。

图 19　值域分类决策树（哪些列写 ES）



数据 · 读图说明见下一页

图 19 • 数据

值域分类决策树（哪些列写 ES）

> 只有「维度列 + `enum` 或低基数 `open`」会逐值写 ES 并置 `sync_values=1`；`range`、主键、外键、度量一律 `sync_values=0`，不写 ES。

这张图回答： `enum` 和 `range` 分别怎么处理？`open` 文本为什么有的进 ES 有的不进？`sync_values` 到底记什么？Meta 的 `examples` 和 ES 的完整值域是什么关系？

读图 1→2→3

1. 先看实体与 grain（一行代表什么）
2. 再看主键/外键与关系
3. 最后看约束、值域与口径

失败：越界时间、缺列、错 grain 会在校验阶段被拒。

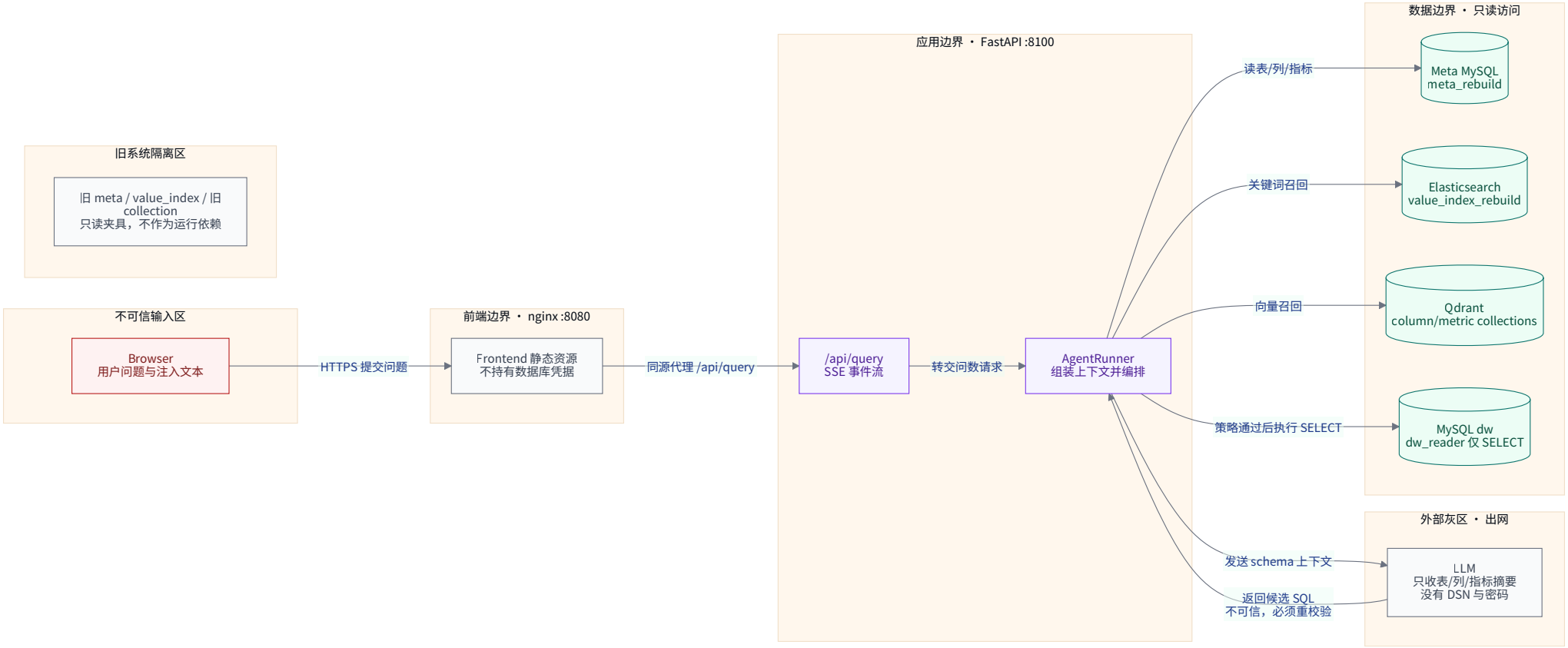
图例：蓝色=判断；橙色=生成 ValueDoc；青色=ES / Meta 字段；灰色=不写 ES 的分支。

关键元素：- 第一问 `role = dimension?`：主键、外键、度量直接 `sync_values=false`。- 第二问 `value_domain.kind`：- `enum`：取 distinct 全集，逐值生成 `ValueDoc`。- `open`：仅当 `0 < distinct ≤ max_value_docs`（默认 1000）才写；`customer_name`（20 值）、`product_name`（15 值）因此进入 ES。- `range` / `id` / `foreign_key`：不写 ES。`date_id`、`month`、`day`、`order_amount` 都在这一类。- 每个 `ValueDoc.id = column_id.value`，写入 `value_index_rebuild`，字段为 `id` / `value` / `column_id`。- `sync_values`：写入 `column_info` 的布尔列，记录「这列的完整取值是否已经进了 ES」。- `examples` 仍只存前 10 条，S3 不能把它当全集。

证据：- `backend/app/build/plan.py` L209–L225：`_should_sync` 的四个条件。- `backend/app/build/plan.py` L312–L341：逐列计算 `sync_values` 并展开 `ValueDoc`。- `backend/app/build/plan.py` L34：`EXAMPLES_LIMIT = 10`。- `backend/app/build/meta_store.py` L36–L48：`sync_values TINYINT(1) NOT NULL DEFAULT 0`。- `backend/app/build/es_store.py` L21–L33：ES mapping 只有 `id` / `value` / `column_id`。- `backend/app/recall/models.py` L16、L92–L93：S3 明确把 `examples` 标为截断。- `backend/app/recall/catalog.py` L187–L192：兜底 payload 里 `examples=[]`、`sync_values=False`。

边界：`sync_values` 目前是**元数据记录**，不是 S3 的运行时开关；S3 读 Meta 的 `examples` 并显式带 `examples_truncated=True`。76 篇 ES 文档是本数据集的结果（enum 41 值 + open 35 值），不是固定常量。`range` 列如果将来需要范围检索，必须改契约与 S2 规则，而不是手工往 ES 塞文档。

图 20 信任边界与攻击面



安全 · 读图说明见下一页

图 20 • 安全

信任边界与攻击面

> 浏览器与 LLM 输出都不可信；前端不持有凭据；FastAPI 是可信计算基；三个存储与 `dw` 都只被只读访问；旧系统只作为夹具，不参与运行。

这张图回答：一次问数跨过哪些信任边界？谁持有数据库凭据？LLM 能拿到什么、拿不到什么？旧系统会不会被新系统写入？

读图 1→2→3

1. 先看信任边界在哪
2. 再按顺序看拦截点
3. 最后看被拒绝的路径与出口

失败：策略拒绝不进入校正，直接 SQL_REJECTED。

图例：红色=不可信输入；紫色=查询时服务；青色=数据存储；灰色=前端 / 外部 LLM / 旧系统边界；虚线=隔离关系。

关键元素：- `Browser`：用户问题、注入文本，全部不可信。- `Frontend`（nginx :8080）：静态资源与同源代理，不含数据库凭据。- `FastAPI` :8100：`/api/query` 与 `AgentRunner`，负责召回、生成、校验、执行。- `LLM`：只接收表/列/指标摘要；**没有 DSN、没有密码**；返回的 SQL 是候选，必须重新过 guard 与 policy。- `Meta MySQL` / `ES` / `Qdrant`：查询时读元知识与索引。- `MySQL dw`：执行器用 `dw_reader`，只有 SELECT。- 旧 `meta` / `value_index` / 旧 collection：只读夹具，不作为运行依赖。

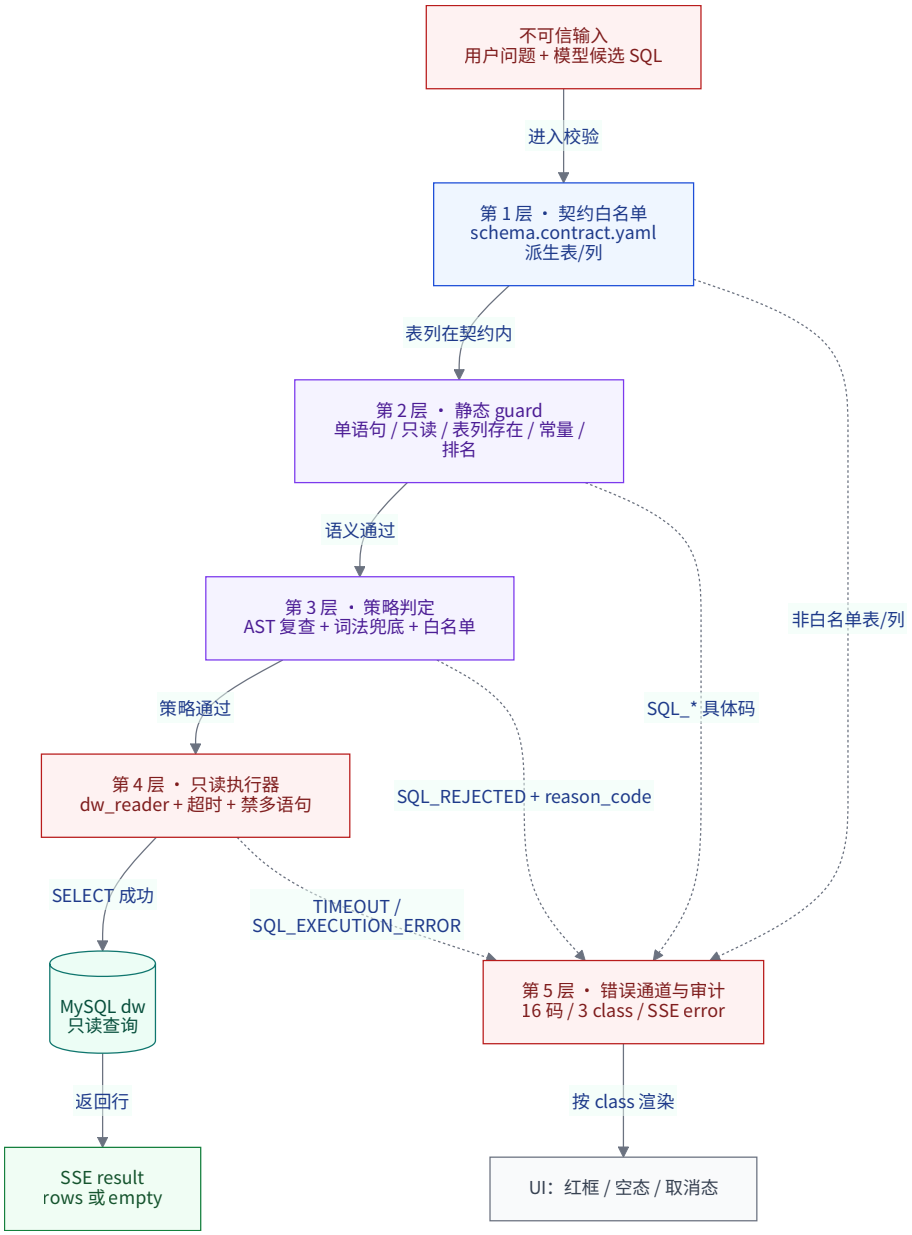
攻击者能做什么、被哪一层挡住、证据在哪：

攻击者动作	被哪一层挡住	证据
在问题里塞 `DROP TABLE` / `UPDATE`	S4 静态 guard 判只读 + S5 策略写节点 + `dw_reader` 无写权限	`backend/app/generate/guard.py` L205–L223；`backend/app/security/policy.py` L390–L411；`contracts/sql.policy.yaml` L44–L58；`backend/app/security/account.py` L31–L47
`SELECT 1; DROP TABLE x` 多语句	策略单语句判定 + 连接标志去掉 `CLIENT_MULTI_STATEMENTS`	`policy.py` L353–L371；`executor.py` L201–L214
`UNION` 把别的表数据并进结果外带	策略禁止集合运算	`policy.py` L375–L388；`sql.policy.yaml` L27–L31
`SELECT ... INTO OUTFILE` / `LOAD_FILE` / `SLEEP`	AST 写节点 + 词法禁词 + 危险函数表	`policy.py` L274–L295、L413–L424；`sql.policy.yaml` L44–L68
用 `/*! ... */` 可执行注释绕过解析器	解析前先拒绝可执行注释	`policy.py` L324–L332；`sql.policy.yaml` L33–L35
从 LLM 或提示词里偷数据库密码	LLM 上下文不含凭据；密码只在执行器环境变量	`executor.py` L32、L193–L214；`backend/app/recall/catalog.py` 只暴露表/列/指标字段
诱导模型引用不存在的表/列（`city`、`price`）	契约白名单 + guard 表列存在检查	`policy.py` L436–L503；`guard.py` L225–L272；`contracts/schema.contract.yaml` L29–L31、L126
让新系统写旧系统数据	rebuild 目标全部 `*_rebuild`；旧库不在建库/删库路径里	`backend/app/build/config.py` L53–L68；`meta_store.py` L202–L210

证据：- `backend/app/security/executor.py` L1–L20：四道闸的设计说明。- `backend/app/security/policy.py` L298–L528：`enforce` 全流程。- `contracts/sql.policy.yaml` L84–L93：执行账号 `dw_reader` 只应有 SELECT。- `fixtures/dw.sql` L9：旧 `didilili` 的 `GRANT ALL PRIVILEGES ON dw.*`，是新系统明确不复用的负向证据。- `design/inventory/facts.md` L103–L108：安全边界摘要。

边界：这张图是信任分区，不是网络拓扑；端口与容器网络见部署图 25–28。真实 LLM 端点未接（⚪），所以「LLM 只拿到 schema 摘要」由代码构造的上下文推导，未做线上抓包验证。

图 21　五层纵深防御



安全 · 读图说明见下一页

图 21 · 安全

五层纵深防御

> 安全由五层各自生效的关卡组成：契约白名单、静态 guard、策略判定、只读执行器、错误通道与审计。

这张图回答：一条 SQL 从模型出来到数据库要过几关？每关挡住什么？为什么说「策略被绕过也还有数据库权限兜底」？失败最终怎么到 UI?

读图 1→2→3

1. 先看信任边界在哪
2. 再按顺序看拦截点
3. 最后看被拒绝的路径与出口

失败：策略拒绝不进入校正，直接 SQL_REJECTED。

图例：蓝色=契约层；紫色=查询时校验；红色=安全/失败层；青色=数据库；绿色=成功结果；虚线=失败路径。

关键元素：- 第 1 层 · 契约白名单：`schema.contract.yaml` 派生表/列集合，策略不从提示词读白名单。 - 第 2 层 · 静态 guard：单语句、只读、表列存在、无常量投影、排名意图的 ORDER BY / LIMIT。 - 第 3 层 · 策略判定：AST 复查 + 集合运算/危险函数 + 词法兜底 + 表列白名单 + 自动 LIMIT。 - 第 4 层 · 只读执行器：`dw_reader` 只有 SELECT、`max_execution_time`、客户端超时、禁多语句。 - 第 5 层 · 错误通道与审计：`ChannelError` 统一形状，16 码 / 3 class，SSE `type=error`。

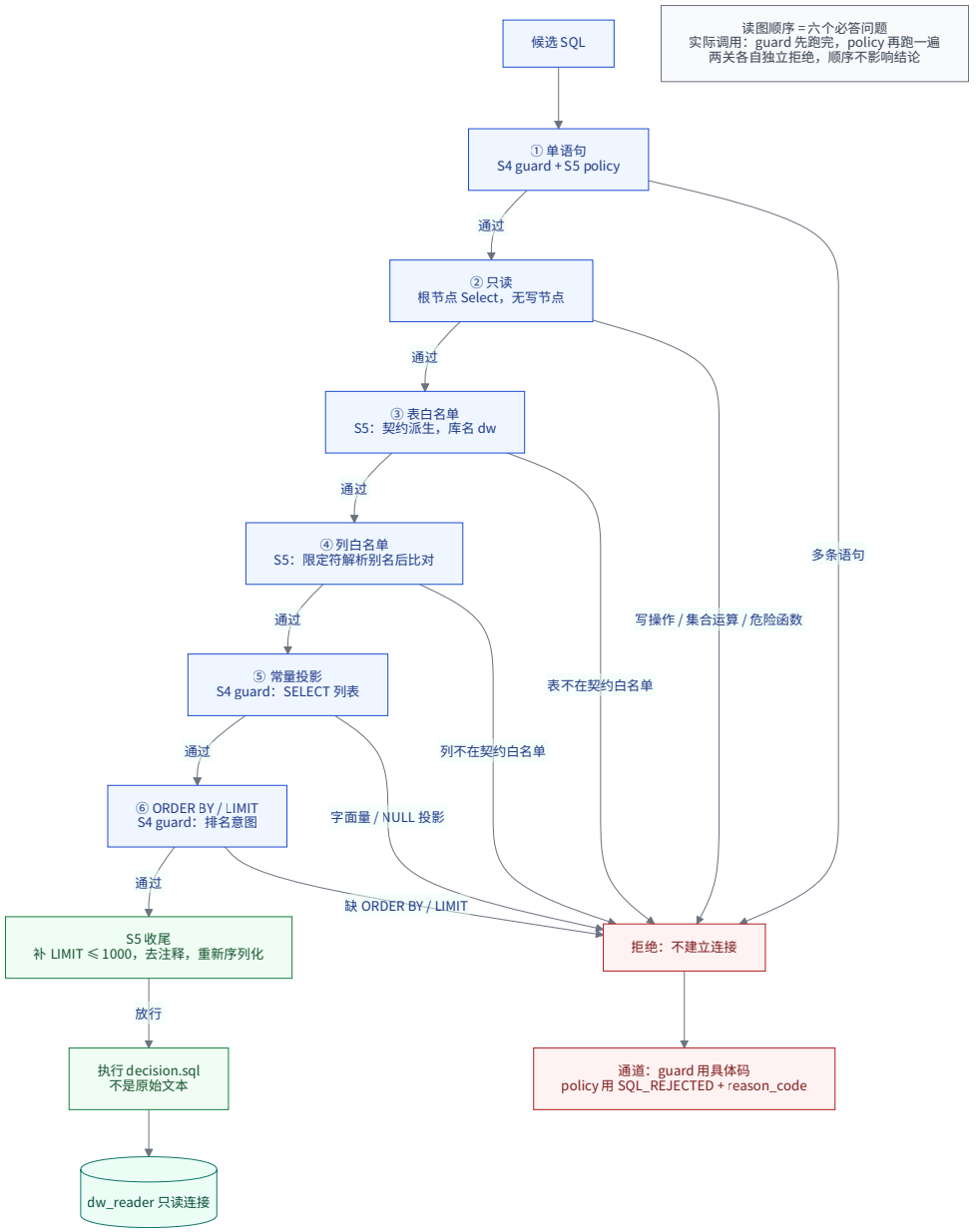
攻击者能做什么、被哪一层挡住、证据在哪：

攻击者动作	挡住它的层	证据
让模型引用契约外的表/列	第 1、2、3 层都会拒绝	`policy.py` L436–L503；`guard.py` L225–L272
生成 `UPDATE` / `DELETE` / DDL	第 2 层只读判定 + 第 3 层写节点	`guard.py` L205–L223；`policy.py` L390–L411
多语句、UNION、可执行注释	第 3 层	`policy.py` L324–L388
	第 3 层禁函数 + 第 4 层 5 秒语句超时	`sql.policy.yaml` L62–L68、L76–L82；
用 `SLEEP` / `BENCHMARK` 拖垮服务		`executor.py` L218–L223
绕过策略直接连库写数据	第 4 层：`dw_reader` 只有 SELECT	`account.py` L31–L47、L142–L190；`contracts/sql.policy.yaml` L84–L93
失败被吞掉、界面全绿	第 5 层：所有失败必须变成 `ChannelError`	`backend/app/agent/errors.py` L274–L350

证据：- `backend/app/agent/validate.py` L1–L70：guard 与 policy 两道关的调用顺序。 - `backend/app/generate/guard.py` L151–L315：静态校验五项。 - `backend/app/security/policy.py` L298–L528：策略十二步。 - `backend/app/security/executor.py` L1–L20、L161–L287：四道闸与执行主流程。 - `backend/app/agent/errors.py` L43–L69、L274–L350：16 码与唯一失败形状。 - `contracts/errors.yaml` L50–L148：码表本身。

边界：五层都依赖同一个契约事实源，契约写错会同时削弱多层。真实数据库权限验证 `--verify-ro-account` 需要 MySQL，本机未跑（⚪），只读结论来自 `account.py` 的静态检查与测试代码。

图 22 SQL 策略判定流程（六关 + 收尾）



安全 · 读图说明见下一页

图 22 • 安全

SQL 策略判定流程（六关 + 收尾）

> 候选 SQL 必须依次回答六个问题——单语句、只读、表白名单、列白名单、无常量投影、排名有 ORDER BY/LIMIT——全过才补 LIMIT 并执行重新序列化后的 SQL。

这张图回答：每一关拒绝什么？拒绝后会不会连库？为什么执行的不是模型返回的原文？guard 和 policy 的先后顺序是什么？

读图 1→2→3

1. 先看信任边界在哪
2. 再按顺序看拦截点
3. 最后看被拒绝的路径与出口

失败：策略拒绝不进入校正，直接 SQL_REJECTED。

图例：蓝色=校验关卡；绿色=放行与执行；红色=拒绝；灰色=读图说明。

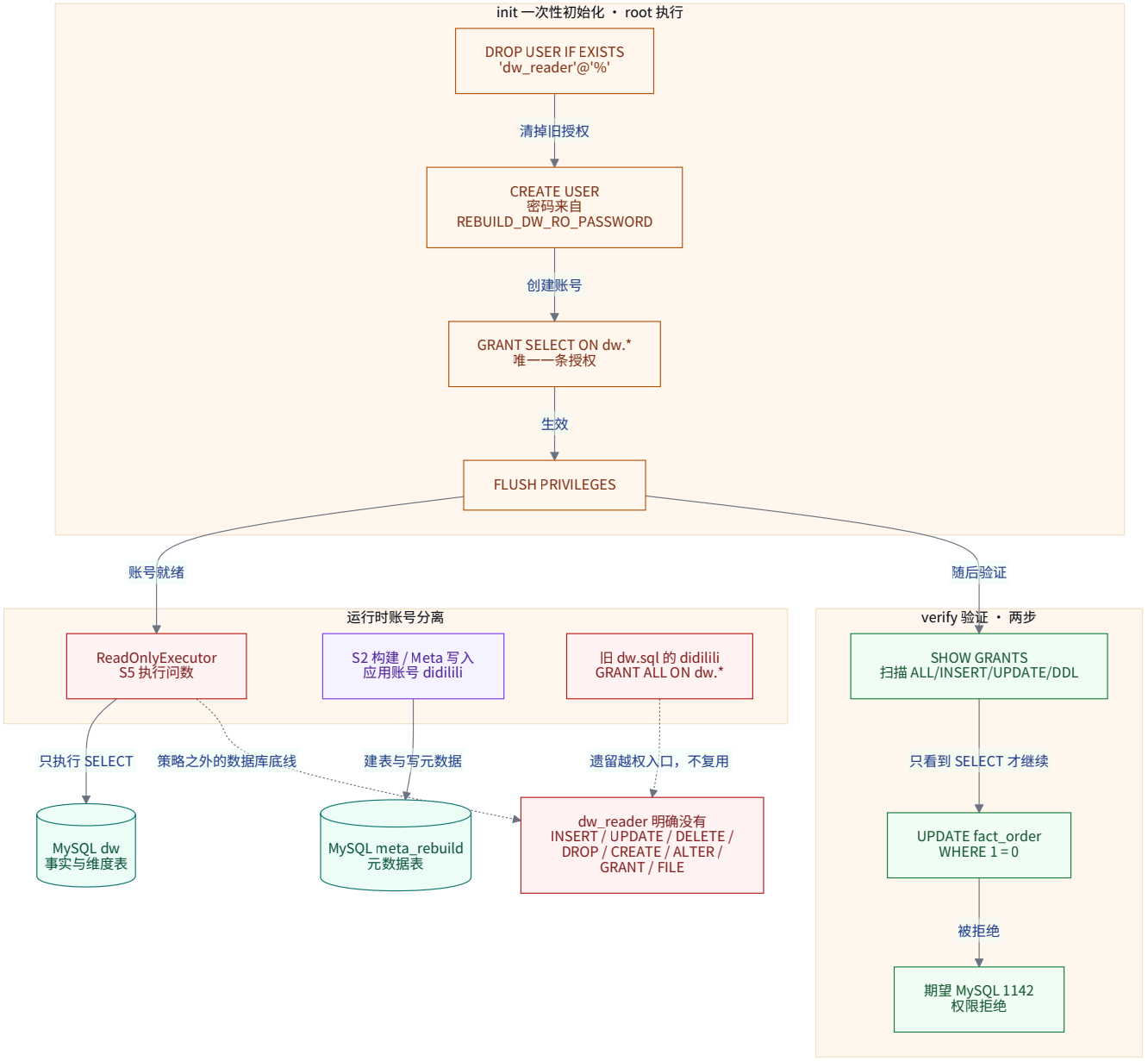
关键元素：- ① 单语句：S4 guard 与 S5 policy 都查；多语句对应 `SQL_MULTIPLE_STATEMENTS`。- ② 只读：根节点必须是 `Select`；任何 DML/DDL/ `INTO` 对应 `SQL_NOT_READ_ONLY`；集合运算与危险函数对应 `SQL_FORBIDDEN_CONSTRUCT`。- ③ 表白名单：S5 用契约派生的 `SqlWhitelist`，库名只允许 `dw`；违规原因码 `SQL_TABLE_NOT_ALLOWED`。- ④ 列白名单：先解析别名，再比对；违规原因码 `SQL_COLUMN_NOT_ALLOWED`。- ⑤ 常量投影：S4 guard 检查 SELECT 列表的字面量或裸 NULL；违规码 `SQL_CONSTANT_PROJECTION`。- ⑥ ORDER BY / LIMIT：问题有排名意图时必须有 ORDER BY；显式「前 N / 最高 / 最低」还必须有 LIMIT；违规码 `SQL_MISSING_ORDER_BY` / `SQL_MISSING_LIMIT`。- 收尾：S5 补 `LIMIT 1000`（超过则截断到 1000），剥掉注释并重新序列化；执行器跑的是 `decision.sql`。

实际调用顺序：`validate_sql` 先跑完整 guard（语义），再跑 policy（安全）；policy 会重新检查单语句与只读，并做白名单与 LIMIT。图中按「六个必答问题」排序，两关各自独立拒绝，顺序不影响最终结论。策略拒绝对外统一为 `SQL_REJECTED`，细因放 `reason_code`；guard 的违规码直接作为通道码。

证据：- `backend/app/agent/validate.py` L31–L70：guard → policy 的调用与错误映射。- `backend/app/generate/guard.py` L151–L315：单语句、只读、表列、常量、排名五段。- `backend/app/security/policy.py` L298–L528：`enforce` 的十二步。- `contracts/sql.policy.yaml` L19–L82：单语句、允许根、禁集合运算、禁可执行注释、禁词、LIMIT、超时。- `contracts/sql.policy.yaml` L94–L125：`SQL_REJECTED` 通道码与 8 个 `reason_codes`。- `backend/app/security/executor.py` L226–L241：策略不通过时直接返回，不连库。- `backend/app/security/policy.py` L508–L528：LIMIT 与重新序列化。

边界：常量投影与 ORDER BY/LIMIT 属于 S4 guard 的**语义**断言，不是 S5 策略；图把它们放在同一条流程里，是因为用户看到的是「校验SQL」一个步骤。列白名单的「无限定符」分支会检查该列是否存在于任一白名单表，因此同名但不同表的歧义列不会被误放行。

图 23　只读账号权限模型（dw_reader 与应用账号分离）



安全 · 读图说明见下一页

图 23 • 安全

只读账号权限模型（dw_reader 与应用账号分离）

> 执行器只用 `dw_reader` 连 `dw`，该账号在 `dw` 上只有一条 `GRANT SELECT`；建库与写 Meta 用应用账号；旧系统的 `didilili` 拥有 `ALL PRIVILEGES`，是新系统明确不复用的负向入口。

这张图回答： `dw_reader` 是谁、怎么创建、怎么验证？执行器和构建任务用的是不是同一个账号？旧 `GRANT ALL` 为什么危险？应用账号与只读账号怎么分离？

读图 1→2→3

1. 先看信任边界在哪
2. 再按顺序看拦截点
3. 最后看被拒绝的路径与出口

失败：策略拒绝不进入校正，直接 SQL_REJECTED。

图例：橙色=一次性初始化；绿色=验证；红色=安全底线与被拒能力；紫色=应用账号；青色=数据库。

关键元素：- 初始化（root）：`DROP USER IF EXISTS 'dw_reader'@'%'` → `CREATE USER ... IDENTIFIED BY '环境变量'` → `GRANT SELECT ON dw.*` → `FLUSH PRIVILEGES`。- 密码只来自 `REBUILD_DW_RO_PASSWORD`，仓库不存秘密。- 验证两步：`SHOW GRANTS` 扫描 `ALL PRIVILEGES / INSERT / UPDATE / DELETE / DROP / CREATE / ALTER / INDEX / REFERENCES / TRIGGER / EXECUTE / FILE / PROCESS / SUPER / SHUTDOWN / GRANT OPTION`；再执行 `UPDATE fact_order ... WHERE 1 = 0`，期望 MySQL 1142 拒绝。- 运行时：`ReadOnlyExecutor` 以 `dw_reader` 执行 SELECT；S2 构建/Meta 写入用应用账号 `didilili` 写 `meta_rebuild`。- `dw_reader` 明确没有 INSERT / UPDATE / DELETE / DROP / CREATE / ALTER / GRANT / FILE。

攻击者能做什么、被哪一层挡住、证据在哪：

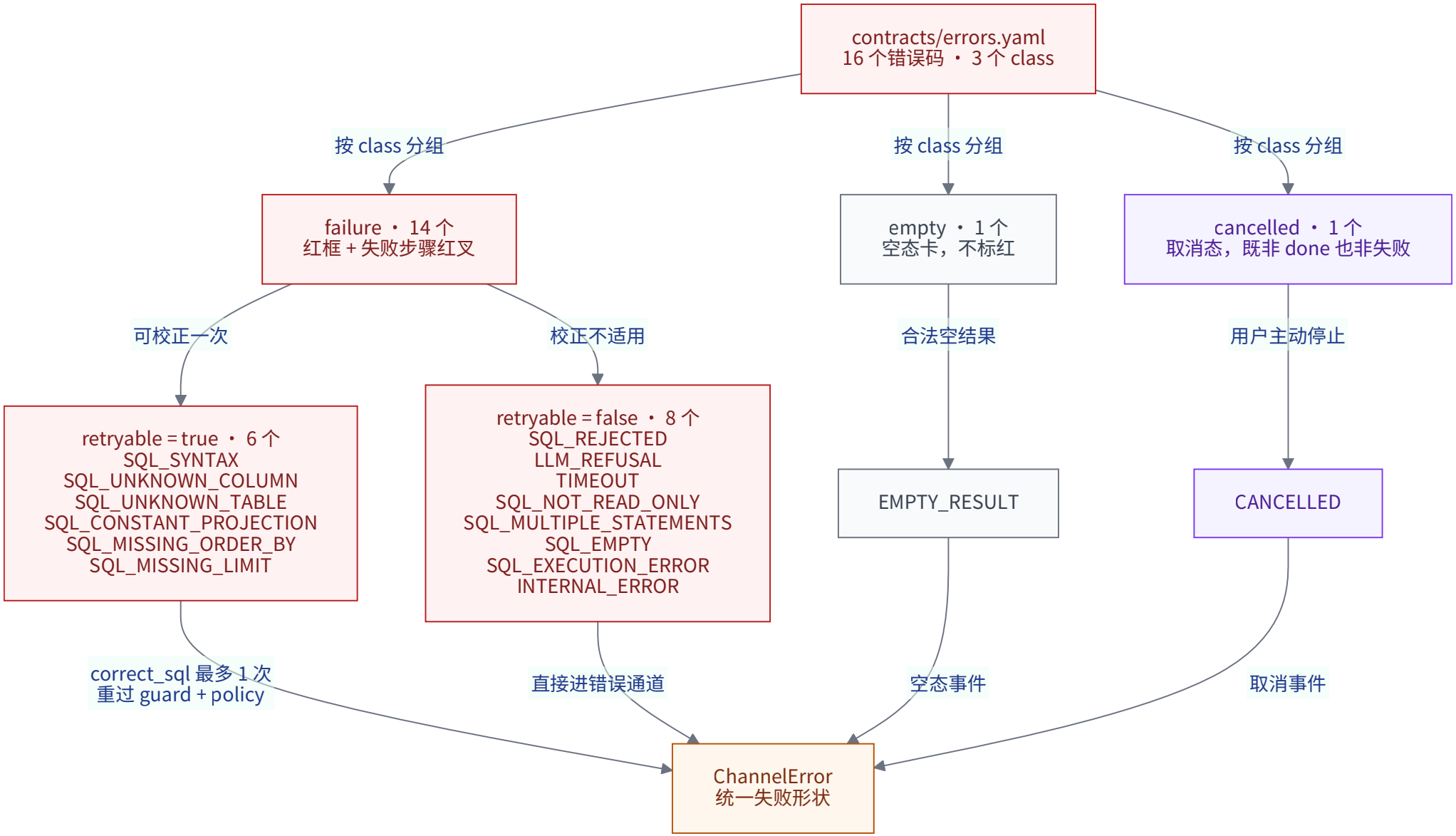
攻击者动作	被哪一层挡住	证据
绕过策略直接执行 `UPDATE` / `DELETE`	数据库权限：`dw_reader` 只有 SELECT，MySQL 返回 1142	`backend/app/security/account.py` L31–L47、L142–L190
建表 / 删表 / 改结构	无 DDL 权限	`contracts/sql.policy.yaml` L84–L93；`account.py` L31–L47
通过 `FILE` 读写服务器文件	无 FILE 权限；策略同时禁 `LOAD_FILE` / `INTO OUTFILE`	`account.py` L31–L47；`sql.policy.yaml` L44–L68
借旧系统 `didilili` 的 ALL 权限写 `dw`	新执行路径不复用该账号；只读账号独立创建	`fixtures/dw.sql` L9；`sql.policy.yaml` L84–L93
从仓库或镜像里找密码	密码只由环境变量提供，不在仓库	`executor.py` L31–L32、L193–L198；`account.py` L65–L70

证据：- `backend/app/security/account.py` L17–L112：`AccountSpec` 与确定性开通语句。- `backend/app/security/account.py` L142–L190：`verify_account` 静态 + 实写验证。- `backend/app/scripts/security.py` L240–L300：`--provision-ro-account` / `--verify-ro-account` CLI。- `backend/app/security/executor.py` L187–L215：只读连接参数与去多语句标志。- `contracts/sql.policy.yaml` L84–L93：账号、库、唯一授权。- `fixtures/dw.sql` L9：旧 `GRANT ALL PRIVILEGES ON dw.* TO 'didilili'@'%'`。- `design/inventory/facts.md` L103–L108：账号分离摘要。

边界：`--verify-ro-account` 需要真实 MySQL，本机未执行（⚪）；本图的开通语句与禁止清单来自代码与契约，未

做线上 `SHOW GRANTS` 抓取。`didilili` 在 `meta_rebuild` 上仍有建表/写入权限，这是应用账号的职责范围，不是只读执行账号。

图 24 错误码分类（16 码 / 3 class / retryable）



安全 · 读图说明见下一页

图 24 • 安全

错误码分类（16 码 / 3 class / retryable）

> `contracts/errors.yaml` 给每一种失败一个名字，并按 `failure` / `empty` / `cancelled` 三类决定 UI；只有 6 个码允许一次 `correct_sql` 校正，`SQL_REJECTED` 永不可重试。

这张图回答：16 个错误码分别属于哪个 class？哪些能自动校正？空结果和用户取消为什么不算失败？失败最后怎么变成 SSE 事件？

读图 1→2→3

1. 先看信任边界在哪
2. 再按顺序看拦截点
3. 最后看被拒绝的路径与出口

失败：策略拒绝不进入校正，直接 SQL_REJECTED。

图例：红色= `failure`；灰色= `empty`；紫色 = `cancelled`；橙色=统一错误通道。节点内列出该组的全部码。

关键元素（16/16）：

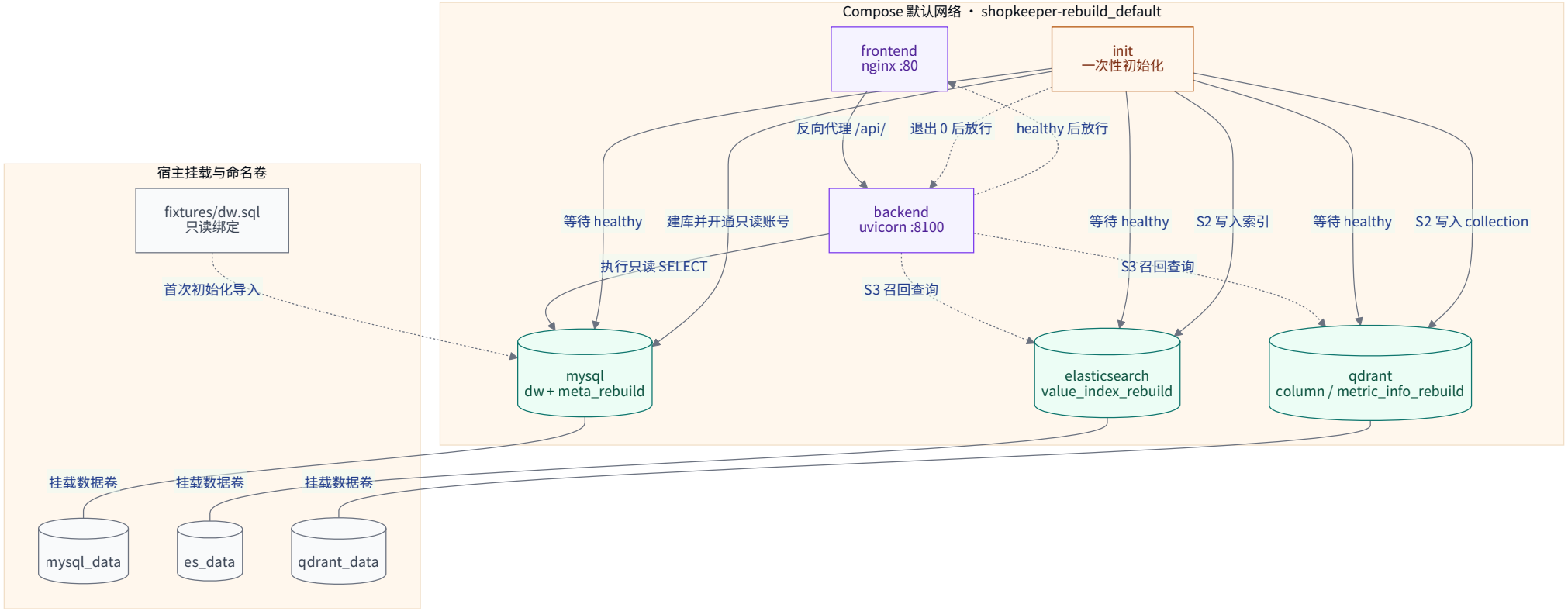
class	retryable	错误码
`failure`	true	`SQL_SYNTAX`、`SQL_UNKNOWN_COLUMN`、`SQL_UNKNOWN_TABLE`、`SQL_CONSTANT_PROJECTION`、`SQL_MISSING_ORDER_BY`、`SQL_MISSING_LIMIT`
`failure`	false	`SQL_REJECTED`、`LLM_REFUSAL`、`TIMEOUT`、`SQL_NOT_READ_ONLY`、`SQL_MULTIPLE_STATEMENTS`、`SQL_EMPTY`、`SQL_EXECUTION_ERROR`、`INTERNAL_ERROR`
`empty`	false	`EMPTY_RESULT`
`cancelled`	false	`CANCELLED`

- `failure`：红框 + 失败步骤红叉，共 14 个。
- `empty`：查询成功但没有数据，空态卡，**不标红**（审计 FP-8），只有 `EMPTY_RESULT`。
- `cancelled`：用户主动停止，取消态，**既不算 done 也不算失败**，只有 `CANCELLED`。
- 校正：`max_attempts=1`，`prompt=correct_sql`，`revalidate=true`，校正后的 SQL 必须重过 `guard` 与 `policy`。
- `SQL_REJECTED` 是策略拒绝的通道码；拿被拒 SQL 去问模型「改一下」等于给注入开重试口子，所以 `retryable=false`。

证据：- `contracts/errors.yaml` L34–L48：三个合法 class 与校正策略。- `contracts/errors.yaml` L50–L148：16 个码的 `label` / `class` / `retryable` / `message`。- `backend/app/agent/errors.py` L155–L220：严格加载、class 校验、路线图 7 码核对。- `backend/app/agent/errors.py` L340–L350：拒答码映射规则（表内保留，表外归 `LLM_REFUSAL`）。- `backend/app/agent/errors.py` L274–L319：`ChannelError` 与 `to_event()`。- `backend/app/agent/correct.py` L144–L232：一次校正、不可重试直接返回、产物校验。

边界：`class` 与 `retryable` 是契约事实，不是 UI 自行推断；`EMPTY_RESULT` 的 `retryable=false` 表示「没有数据」不应自动重试。`ChannelError.klass` 在运行时再查表，若码不在表里会抛 `ErrorTableError`，不会静默降级成 `INTERNAL_ERROR`。

图 25 Docker Compose 拓扑



部署 · 读图说明见下一页

图 25 · 部署

Docker Compose 拓扑

> `compose.yaml` 用一条默认网络把六个服务连起来，三个命名卷承载状态，`init` 是唯一的一次性任务。

这张图回答：哪六个服务？谁依赖谁？数据落在哪？`init` 为什么和另外五个不一样？

读图 1→2→3

1. 先看服务与端口
2. 再看启动依赖顺序
3. 最后看健康检查与失败停点

失败：前置服务未健康则后续不启动，停点写在健康检查里。

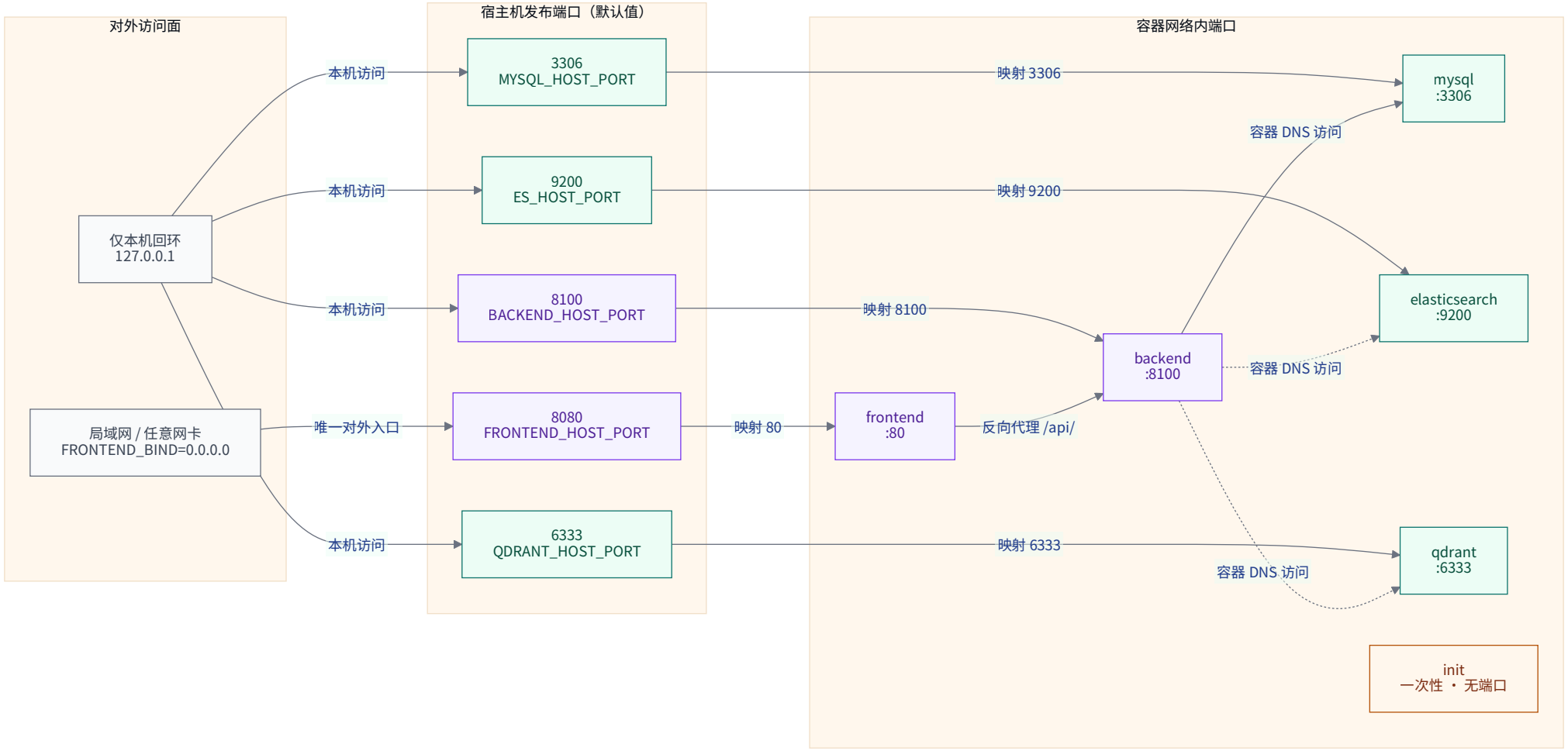
图例：橙色=`init` 一次性构建任务；青色=三存储与卷；紫色=`backend` / `frontend` 查询时服务；灰色=宿主挂载与命名卷。

关键元素：- 三存储：`mysql`、`elasticsearch`、`qdrant`，各自带 healthcheck。- `init`：与 `backend` 共用镜像 `shopkeeper-rebuild/backend:local`，但 `restart: "no"`，正常退出后不再启动。- `backend`：只拿 `dw_reader` 只读凭据；`frontend`：nginx，唯一对外入口。- 依赖方向：三存储 healthy → `init` 启动；`init` 退出 0 → `backend` 启动；`backend` healthy → `frontend` 启动。- 默认网络未显式声明，Compose 按项目名生成 `shopkeeper-rebuild_default`；项目名来自 `compose.yaml` 的 `name:`（可被 `COMPOSE_PROJECT_NAME` 覆盖，卷名与网络名随之变化）。- 命名卷：`mysql_data`、`es_data`、`qdrant_data`；另有一个只读绑定 `./fixtures/dw.sql` → MySQL 初始化目录。

证据：- `compose.yaml` L1（项目名）、L11–L168（六服务）、L98–L104（`init` 依赖三存储 healthy）、L131–L133（`backend` 等 `init` 完成）、L158–L160（`frontend` 等 `backend` healthy）、L170–L173（三个卷）。- `compose.yaml` L28（`fixtures/dw.sql` 只读挂载到 `docker-entrypoint-initdb.d`）。- `docs/DOCKER-DEPLOY.md` §5：卷实际名称为 `shopkeeper-rebuild_mysql_data` / `_es_data` / `_qdrant_data`。- `docs/DOCKER-HANDOFF.md` §1：拓扑与一次性 `init` 的既定事实。

边界：本工作树 Docker socket 无权限（`docker ps` 报 `permission denied`），未实际启动该拓扑，标 ⚪；服务名、卷名、依赖条件均逐字取自 `compose.yaml`。图中不画镜像仓库、跨主机网络与高可用——`docs/DOCKER-DEPLOY.md` §7 明确不提供。

图 26　宿主端口与容器端口暴露面



部署 · 读图说明见下一页

图 26 • 部署

宿主端口与容器端口暴露面

> 默认只有 `frontend` 的 `8080 → 80` 监听所有网卡，其余四个发布端口全部锁在 `127.0.0.1`，`init` 没有宿主端口。

这张图回答：谁能从局域网进来？哪些端口只能本机访问？容器之间走什么端口？

读图 1→2→3

1. 先看服务与端口
2. 再看启动依赖顺序
3. 最后看健康检查与失败停点

失败：前置服务未健康则后续不启动，停点写在健康检查里。

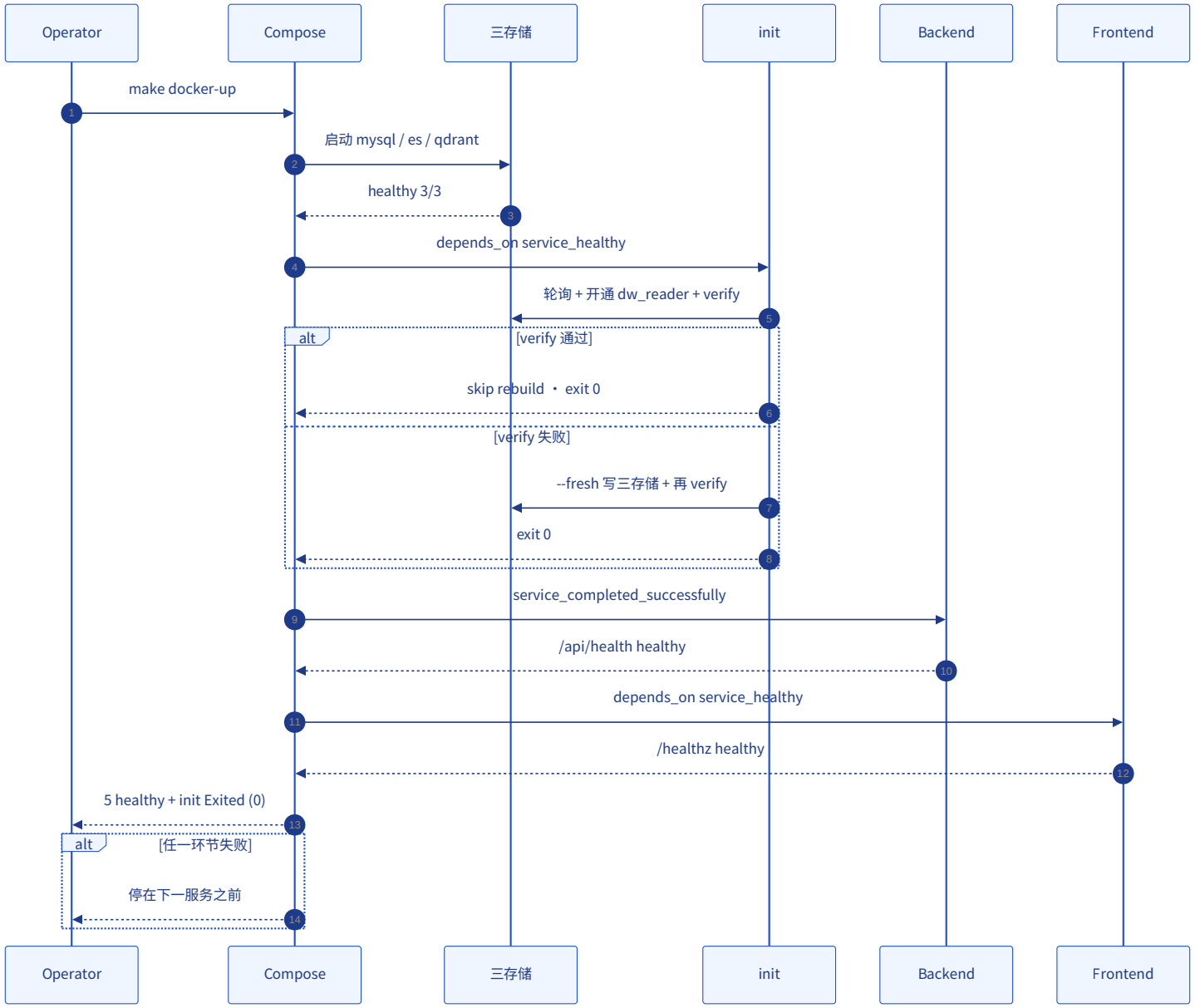
图例：紫色=查询时服务；青色=三存储；橙色=一次性 `init`；灰色=访问面边界。

关键元素：- `frontend`：`${FRONTEND_BIND:-0.0.0.0}:${FRONTEND_HOST_PORT:-8080}:80`，默认 `0.0.0.0`，是唯一对外入口；要收紧就设 `FRONTEND_BIND=127.0.0.1`。- `backend`：`127.0.0.1:${BACKEND_HOST_PORT:-8100}:8100`。- `mysql`：`127.0.0.1:${MYSQL_HOST_PORT:-3306}:3306`。- `elasticsearch`：`127.0.0.1:${ES_HOST_PORT:-9200}:9200`。- `qdrant`：`127.0.0.1:${QDRANT_HOST_PORT:-6333}:6333`。- `init`：无 `ports:`；只能 `docker compose run --rm --no-deps init` 进入，或由 `up` 自动执行一次。- 容器内：`frontend` 通过 `backend:8100` 反向代理 `/api/`；`backend` 通过容器 DNS 访问三存储。

证据：- `compose.yaml` L24–L25（MySQL）、L45–L46（ES）、L63–L64（Qdrant）、L129–L130（backend）、L155–L157（frontend，含 `FRONTEND_BIND` 注释）、L83–L104（`init` 无 `ports`）。- `docker/nginx.conf`：`listen 80`、`location /api/` 代理到 `http://backend:8100`。- `docs/DOCKER-DEPLOY.md` §1：「只有前端 `8080` 监听所有网卡，其余端口都只绑定 `127.0.0.1`」。- `docs/DOCKER-DEPLOY.md` §2.1：容器 `backend` 与本机 `make dev-backend` 都占 `8100`，不能同时跑。

边界：端口映射是文件事实（✅），但本工作树无 Docker 权限，未用 `docker compose port` 复核实际发布结果（⚪）。`FRONTEND_BIND` 默认 `0.0.0.0` 且无鉴权，是文档写明的取舍（`docs/DOCKER-HANDOFF.md` §3 B11）。

图 27　启动顺序与失败停点



部署 · 读图说明见下一页

图 27 • 部署

启动顺序与失败停点

> `docker compose up` 是一道四级闸门——三存储 healthy → `init` 退出 0 → `backend` healthy → `frontend` 启动；任何一级不过，后面的服务都不会起来。

这张图回答：谁在等谁？每级等多久？失败会停在哪一步？`init` 内部又做了什么？

读图 1→2→3

1. 先看服务与端口
2. 再看启动依赖顺序
3. 最后看健康检查与失败停点

失败：前置服务未健康则后续不启动，停点写在健康检查里。

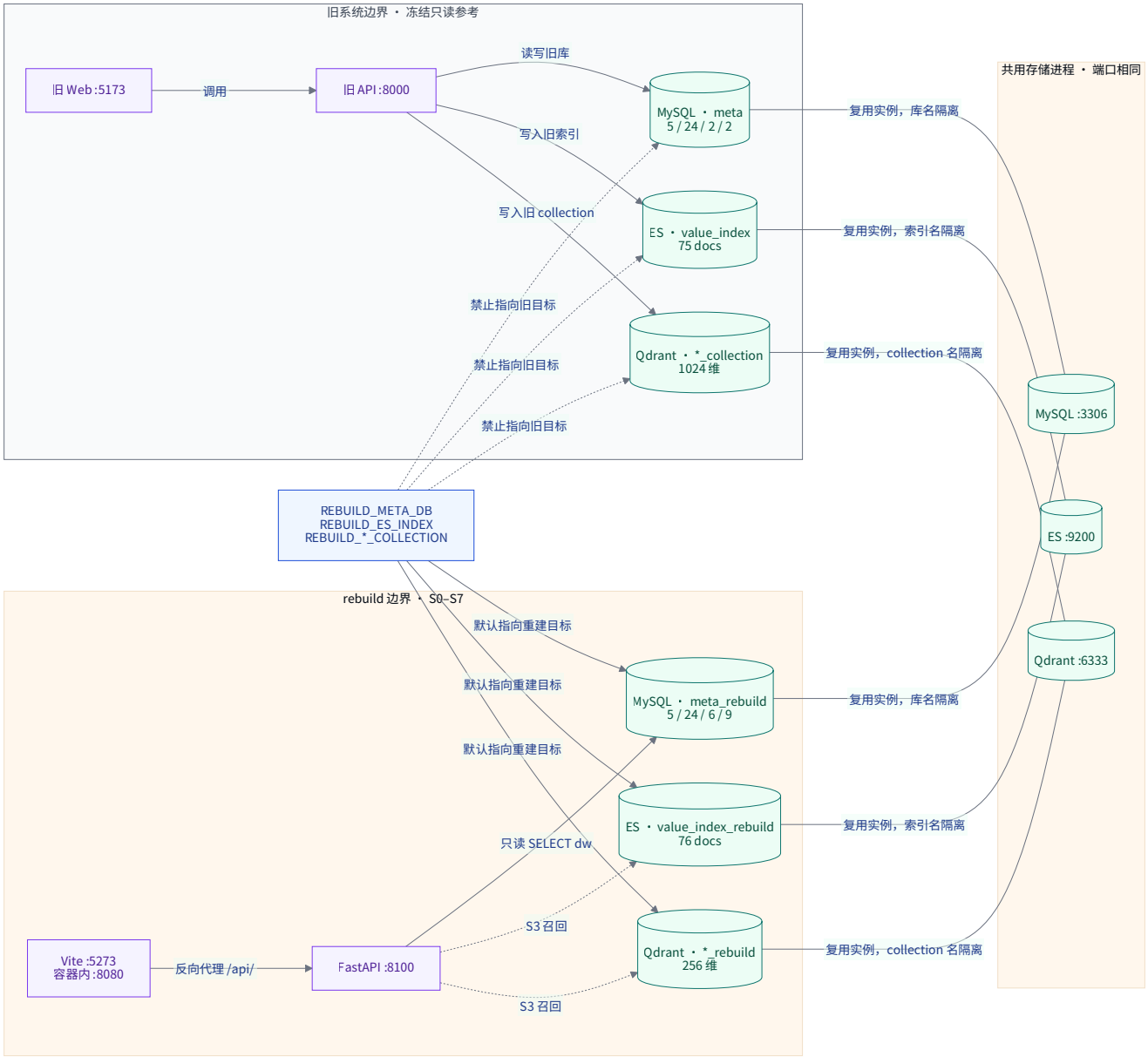
图例：参与者用真实进程名；`init` 内部按 `docker_init.py` 的实际调用顺序展开。

关键元素：- MySQL healthcheck：`mysqladmin ping`，`5s × 60`，`start_period 30s`。- Elasticsearch healthcheck：`_cluster/health?wait_for_status=yellow`，`10s × 30`，`start_period 30s`。- Qdrant healthcheck：`/readyz` 返回 200，`10s × 30`，`start_period 10s`（要求 Qdrant ≥ v1.5.0）。- `init`：`depends_on` 三存储 `service_healthy`；内部再轮询三项（每项最多 180s），随后开通 `dw_reader`、验证只读、`--verify` 元知识；校验失败才 `--fresh` 重建并再次 `--verify`。- `backend`：`depends_on init: service_completed_successfully`；只有 `init` 退出 0 才启动。- `frontend`：`depends_on backend: service_healthy`。- 失败停点：存储不 healthy → 停在 `init` 前；`init` 非 0 → 停在 `backend` 前；`backend` 不 healthy → 停在 `frontend` 前。

证据：- `compose.yaml` L29–L36（MySQL）、L49–L58（ES）、L67–L81（Qdrant）、L98–L104（`init`）、L131–L143（`backend`）、L158–L168（`frontend`）。- `backend/app/scripts/docker_init.py` L41–L118：`wait_for_services` → `initialize`（开通只读账号 → 验证 → `--verify` → 必要时 `--fresh` → 再 `--verify`）。- `docs/DOCKER-DEPLOY.md` §2：五步等待顺序与「`init` 退出 0 后 backend 才启动」。- `docs/DOCKER-DEPLOY.md` §7：backend 固定单 worker（取消注册表是进程内状态）。

边界：顺序与超时是 `compose.yaml` / `docker_init.py` 的文件事实（✅）；本工作树没有 Docker socket 权限，未实际观察过一次 `up` 的完整时序（⚪）。`init` 的「跳过重建」路径在 `docs/DOCKER-HANDOFF.md` §6.1 有他机 smoke 记录（🟡，非本工作树复跑）。

图 28　旧系统与 rebuild 的隔离边界



部署 · 读图说明见下一页

图 28 • 部署

旧系统与 rebuild 的隔离边界

> 旧系统与 rebuild 共用同一批存储进程，但靠库名、索引名、collection 名与向量维度隔离；S2/S3 的默认目标一律带 `_rebuild`。

这张图回答：旧系统的 `meta` / `value_index` / `5173` / `3306` 与 rebuild 的 `*_rebuild` / `5273` / `8100` 在哪一层分开？哪些东西其实是共用的？

读图 1→2→3

1. 先看服务与端口
2. 再看启动依赖顺序
3. 最后看健康检查与失败停点

失败：前置服务未健康则后续不启动，停点写在健康检查里。

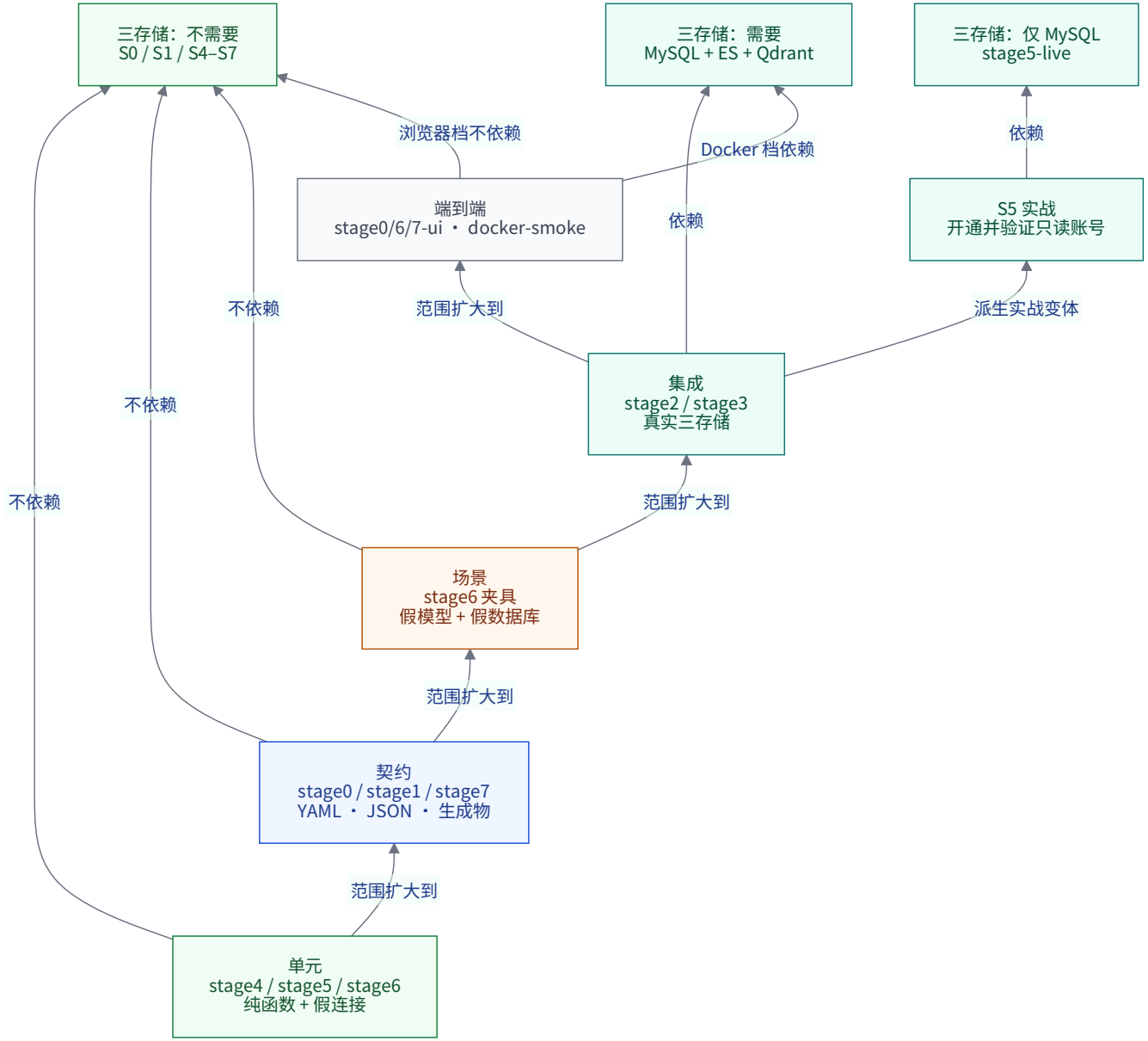
图例：蓝色=可配置的目标名（契约式入口）；青色=存储与目标；紫色=服务；灰色=旧系统边界。

关键元素：- 旧系统：Web `5173`、API `8000`、MySQL 库 `meta`（5 / 24 / 2 / 2）、ES 索引 `value_index`（75 docs）、Qdrant `column_info_collection` 与 `metric_info_collection`（1024 维）。- rebuild：Vite `5273`（容器内 nginx `8080`）、FastAPI `8100`、MySQL `meta_rebuild`（5 / 24 / 6 / 9）、ES `value_index_rebuild`（76 docs）、Qdrant `column_info_rebuild` / `metric_info_rebuild`（256 维）。- 共用存储进程：如果本机已经在 `3306 / 9200 / 6333` 上跑着旧系统的存储，rebuild 直接探活复用，不重复 `up`；此时隔离靠**名字**而不是端口。- 目标入口：`REBUILD_META_DB`、`REBUILD_ES_INDEX`、`REBUILD_COLUMN_COLLECTION`、`REBUILD_METRIC_COLLECTION`，默认值全部带 `_rebuild`。- 红线：绝对不要把目标指到 `meta` / `value_index` / `*_collection`；S2 的 `--fresh` 会重建目标。

证据：- `backend/app/build/config.py` 的默认值（`meta_rebuild` / `value_index_rebuild` / `column_info_rebuild` / `metric_info_rebuild`，见 `docs/SETUP-FROM-SCRATCH.md` §4 表）。- `docs/SETUP-FROM-SCRATCH.md` §5：旧系统与 rebuild 的计数对照，以及「执行 `make stage2` / `make stage3` 后旧系统计数没有变化」（✅本机 2026-10-25）。- `docs/SETUP-FROM-SCRATCH.md` §4「自定义目标时，S2 和 S3 必须用同一组变量」L918–L962。- `README.md` L247–L248、L254–L255：只动 `*_rebuild`；端口隔离与旧系统 `8000/5173` 可同时运行。- `docs/DOCKER-DEPLOY.md` §5：Compose 卷名带项目名前缀，`down -v` 只影响 rebuild 项目。

边界：名字与默认值是文件事实（✅）；「旧系统计数未变」是他机/前一轮本机记录（🟡，本次未复跑）。`3306` 不是隔离出来的：完整 Docker 部署把 MySQL 发布到 `127.0.0.1:3306`，与旧系统 MySQL 同端口时只能二选一；能共存的是 `8100 / 5273` 与旧系统 `8000 / 5173`。

图 29　测试金字塔与三存储依赖



测试 · 读图说明见下一页

图 29 • 测试

测试金字塔与三存储依赖

> 绝大多数测试离线可跑；需要三存储的只有 S2 构建、S3 召回和 Docker 全栈 smoke，S5 实战只要 MySQL。

这张图回答：五层测试各自覆盖什么？哪一层开始需要 MySQL / ES / Qdrant？哪些是「本机就能重复」的门禁？

读图 1→2→3

1. 先看层级（单元/集成/端到端）
2. 再看阶段门禁顺序
3. 最后看失败注入覆盖了哪些码

失败：任一阶段门禁红，不能进入下一阶段。

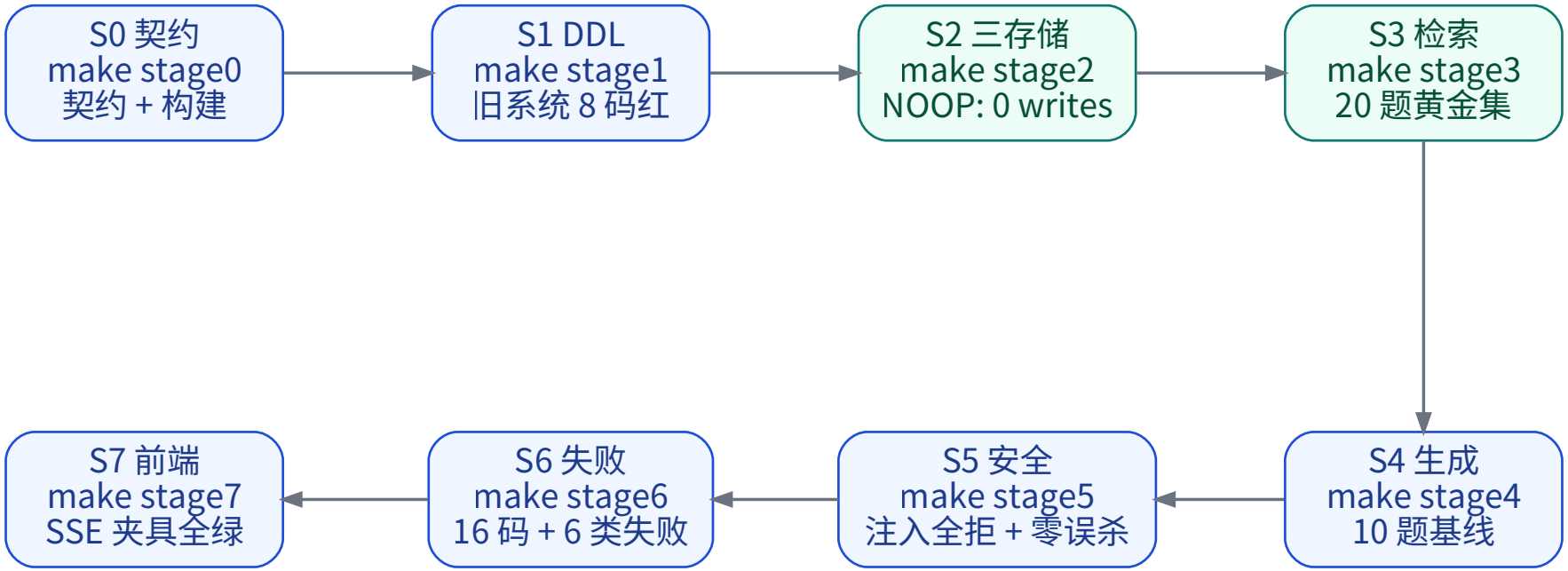
图例：绿色=无需三存储；青色=需要三存储；橙色=夹具驱动的场景测试；蓝色=契约；灰色=浏览器 / 全栈端到端边界。

关键元素：- 单元：`tests/generate`、`tests/security`、`tests/agent`（不含 scenarios）、`tests/recall` 的纯逻辑测试、顶层脚本测试（`test_docker_*`、`test_smoke_checks` 等）——用假连接或纯函数，不连存储。- 契约：`tests/test_s0_contract.py`、`tests/test_s1_schema_contract.py`、`tests/frontend/` 的生成物 / SSE 夹具 / 类型保真，以及 `test_contract_alignment`、`test_error_alignment`。- 场景：`tests/agent/test_scenarios.py` 跑 `fixtures/failures/scenarios.yaml`；只替换模型与数据库，校验、策略、校正、SSE 都是真代码。- 集成：`tests/recall/test_golden.py` 的真实检索用例必须 `REBUILD_INTEGRATION=1`；否则跳过（文档口径：31 条跳过里 23 条是 S3 真实存储集成）。- 端到端：`stage0-ui` / `stage6-ui` / `stage7-ui` 自起临时后端与前端（8101/5274、8102/5275、8103/5276），不需要三存储；`docker-smoke` 需要整套 Compose。- S5 实战：`make stage5-live` 只要 MySQL，用于开通并验证 `dw_reader`。

证据：- `README.md` L28–L29：`stage2` / `stage3` 需要三存储；`stage4`–`stage7` 完全离线；UI 门禁需要 Chrome 或 Edge。- `Makefile` L38–L64（S2 / S3 前置）、L67–L76（S4 明确不需要三存储）、L83–L106（S5 离线门禁与 `stage5-live`）。- `backend/tests/recall/test_golden.py` L111–L125：`REBUILD_INTEGRATION=1` 才跑真实 ES/Qdrant/Meta。- `scripts/stage0-ui.sh` L10–L12、`scripts/stage6-ui.sh` L21–L23、`scripts/stage7-ui.sh` L21–L23：临时端口，避开 dev 服务。- `docs/SETUP-FROM-SCRATCH.md` §3 表 L835–L847：每阶段是否需要三存储。- `backend/tests/` 目录结构（本次 `find` 实际读取）。

边界：本工作树只做了图源与渲染，没有跑任何测试；「782 passed / 31 skipped」来自 README / HANDOFF（🟡），`SETUP-FROM-SCRATCH.md` §3 另写「665 passed / 31 skipped」（🟡，已过期，两处口径不一致）。真实三存储集成与 Docker smoke 在本机标 ⚪。

图 30 S0–S7 阶段门禁



测试 · 读图说明见下一页

图 30 • 测试

S0–S7 阶段门禁

> 每个阶段都有一条 `make stageN` 硬命令；命令最后打印 `S0 PASS`…`S7 PASS` 才算过，PASS 后才允许进入下一阶段。

这张图回答：每个阶段跑什么？「通过」的判据是什么？哪两个阶段需要先起三存储？

读图 1→2→3

1. 先看层级（单元/集成/端到端）
2. 再看阶段门禁顺序
3. 最后看失败注入覆盖了哪些码

失败：任一阶段门禁红，不能进入下一阶段。

图例：蓝色=离线门禁；青色=需要三存储（S2 / S3）。

关键元素：

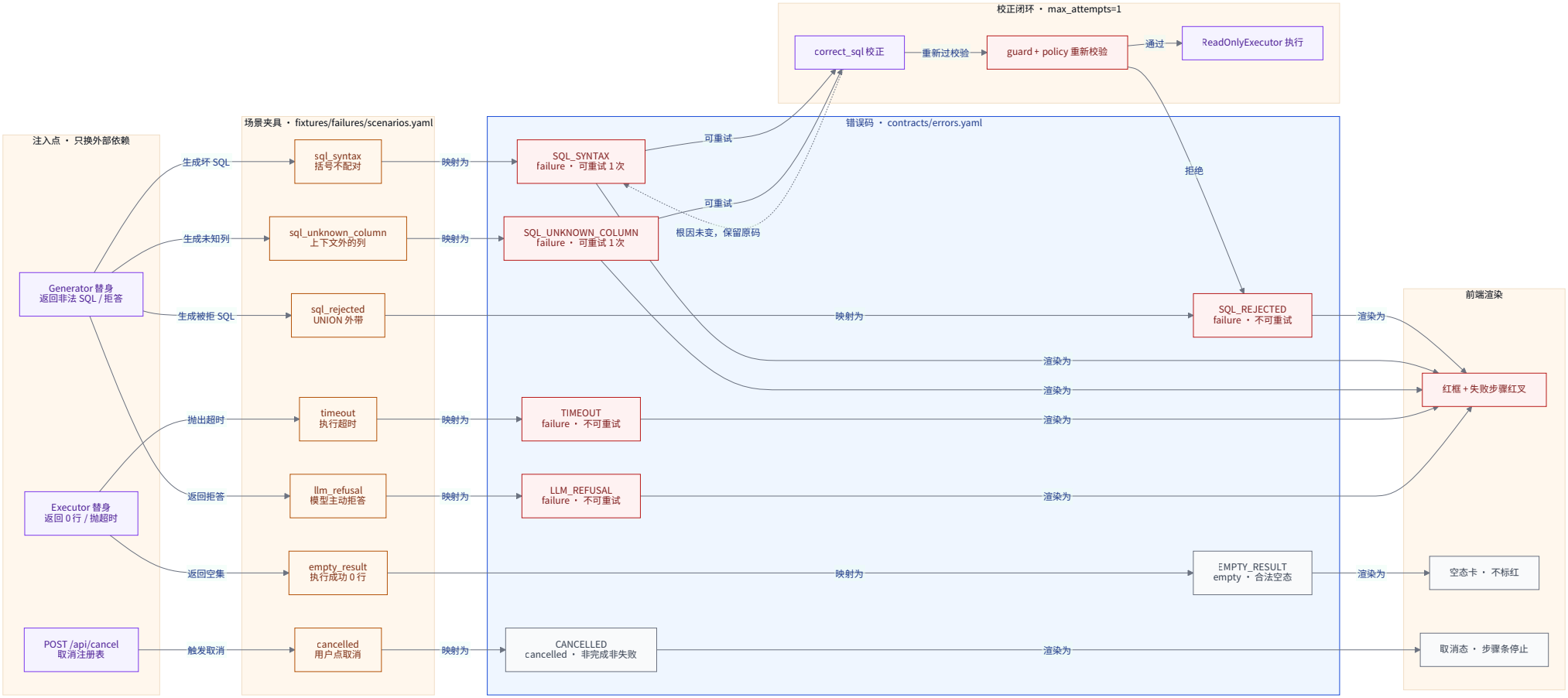
阶段	硬验收命令	「通过」判据
S0	`make stage0`	契约测试 + 前端单测 + 前端构建全绿，打印 `S0 PASS`
S1	`make stage1`	契约 ↔ DDL ↔ 数据全绿；旧系统负向夹具必须红；打印 `S1 PASS`
S2	`make stage2`	`--dry-run` → `--fresh` → `--verify` 通过；`--assert-idempotent` 输出 `NOOP: 0 writes`；打印 `S2 PASS`
S3	`make stage3`	S2 目标 `--verify` 通过；20 题黄金集集成测试通过；CLI 产出 YAML；打印 `S3 PASS`
S4	`make stage4`	10 题基线全过 + 静态校验；生成层测试全绿；打印 `S4 PASS`
S5	`make stage5`	注入语料 100% 拒绝 + 零误杀；安全层测试全绿；打印 `S5 PASS`
S6	`make stage6`	16 个错误码 + 6 类失败 + 取消；失败层与前端 SSE / 构建全绿；打印 `S6 PASS`
S7	`make stage7`	生成物新鲜度 + SSE 夹具逐字对账 + 前端契约 / 单测 / 构建全绿；打印 `S7 PASS`

- 可选变体：`stage0-ui` / `stage6-ui` / `stage7-ui` 需要 Chrome 或 Edge；`stage5-live` 只需要 MySQL 与 `REBUILD_DW_RO_PASSWORD`。
- S2 / S3 之外的阶段都可以在没有任何存储的机器上重复；S2 / S3 的前置写在 `Makefile` 注释里。

证据：- `Makefile` L14–L25（S0）、L28–L35（S1）、L38–L50（S2）、L53–L64（S3）、L67–L76（S4）、L83–L106（S5 与 live）、L112–L133（S6 与 UI）、L138–L157（S7 与 UI）。- `docs/SETUP-FROM-SCRATCH.md` §3 L835–L847：阶段 / 命令 / 是否需要三存储 / 核心验收。- `README.md` L15–L23：S0–S7 的验收摘要；L25：当前测试口径。- `backend/tests/` 的子目录与 `scripts/stage*-ui.sh` 的实际端口。

边界：所有 `stageN` 目标与判据都能在 `Makefile` 里逐字指认（✅文件事实）；但「某次运行打印了 S0 PASS」属于他机记录，本工作树**没有**重跑，标 🟡。测试计数存在口径冲突：`README.md` / `docs/DOCKER-HANDOFF.md` 说 782 passed，`docs/SETUP-FROM-SCRATCH.md` §3 说 665 passed——图里因而不写死总测试数，只写判据。S8 尚未实现（⚪）。

图 31 失败注入矩阵



测试 · 读图说明见下一页

图 31 · 测试

失败注入矩阵

> 六类失败只靠两个替身注入——假模型与假数据库；校验、安全策略、校正闭环和 SSE 都是真代码，所以每一类失败都能映射到一个确定的错误码。

这张图回答：失败从哪里注入？落到哪个错误码？属于 `failure` / `empty` / `cancelled` 哪一类？哪些会触发校正闭环？界面上长什么样？

读图 1→2→3

1. 先看层级（单元/集成/端到端）
2. 再看阶段门禁顺序
3. 最后看失败注入覆盖了哪些码

失败：任一阶段门禁红，不能进入下一阶段。

图例：红色=失败与安全拒绝；橙色=夹具场景；紫色=注入替身与执行器；灰色=合法空态与用户取消；蓝色=错误码契约。

关键元素：

场景 id	注入点	错误码	class	可重试	结果	证据
`sql_rejected`	Generator 返回 UNION 外带	`SQL_REJECTED`	failure	否	run_sql 被拒	红框 + 红叉
`sql_syntax`	Generator 返回括号不配对	`SQL_SYNTAX`	failure	是（1 次）	validate_sql 失败	红框 + 红叉
`sql_unknown_column`	Generator 引用上下文外的列	`SQL_UNKNOWN_COLUMN`	failure	是（1 次）	validate_sql 失败	红框 + 红叉
`empty_result`	Executor 返回 0 行	`EMPTY_RESULT`	empty	否	run_sql 空态卡，不计	橙框
`timeout`	Executor 抛超时	`TIMEOUT`	failure	否	run_sql 超时	红框 + 红叉
`llm_refusal`	Generator 主动拒答	`LLM_REFUSAL`	failure	否	generate_sql 拒绝	红框 + 红叉
取消按钮	`POST /api/`	`CANCELLED`	cancelled	否	用户取消	灰框

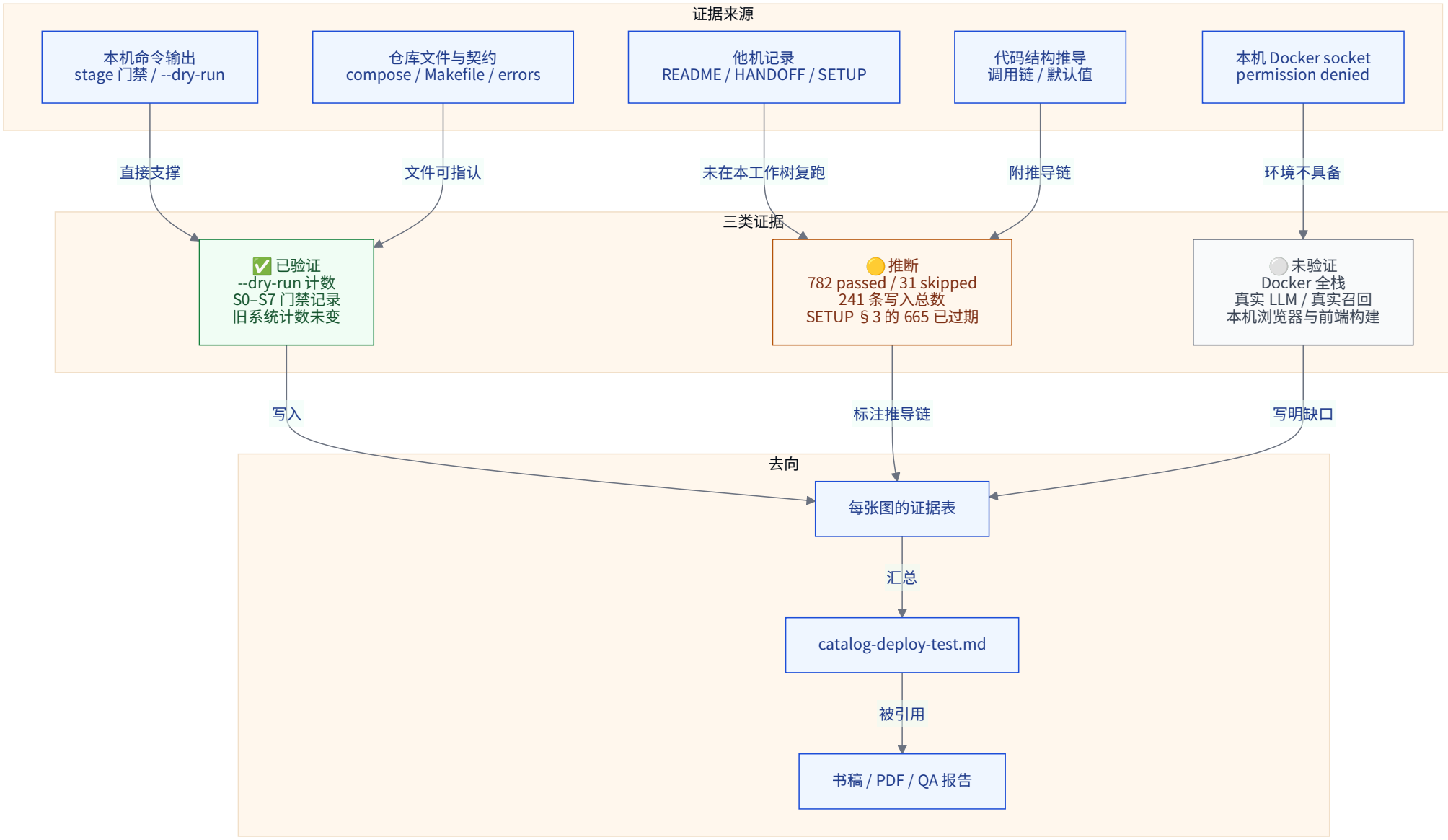
校正闭环：`SQL_SYNTAX` / `SQL_UNKNOWN_COLUMN` 可重试；`correct_sql` 最多 1 次，校正后的 SQL 重新过 `guard + policy`；被拒绝回 `SQL_REJECTED`。

- 安全红线：`SQL_REJECTED` 永远不可重试——拿被拒的 SQL 去问模型「改一下」等于给注入开重试口子。

证据：- `fixtures/failures/scenarios.yaml` L51–L73（`sql_rejected`）、L75–L94（`sql_syntax`）、L96–L111（`sql_unknown_column`）、L113–L132（`empty_result`）、L134–L150（`llm_refusal`）、L152–L171（`timeout`）、L173–L191（`slow`，供取消演练）。- `contracts/errors.yaml` L42–L49（`correction.max_attempts=1`、`revalidate_with: guard + policy`）、L52–L92（七个通道码与 `class` / `retryable`）。- `backend/tests/agent/test_scenarios.py` L20–L30：6 类失败场景清单；每个场景实际跑一遍流水线并与 `expect_*` 对账。- `Makefile` L108–L133：`stage6` 的 `--check` / `--table` / `--scenarios` + `tests/agent` + 前端 SSE / 构建。- `docs/SETUP-FROM-SCRATCH.md` §2.9 L608–L665：场景表与期望；§7 记录 `stage6-ui` 在真实 Edge 上给出 6 种不同状态。

边界：场景测试只替换模型与数据库，不需要 MySQL / LLM（✅ `Makefile` 注释与 `tests/agent/conftest.py`）；但本工作树没有重跑它们（🟡）。浏览器验收 `stage6-ui` 需要 Chrome / Edge，本次未跑（⚪）。`CANCELLED` 没有场景夹具——它由取消接口在慢查询上触发。

图 32 证据台账：已验证 / 推断 / 未验证



测试 • 读图说明见下一页

图 32 • 测试

证据台账：已验证 / 推断 / 未验证

> 每条结论只允许三种状态——本机跑过（✅）、从结构推导（🟡）、环境不具备（⚪）；三类证据都要写进图说明，缺口也要写出来。

这张图回答：图里的结论从哪来？哪些能在本机复现？哪些只是引用他机记录？最终汇到哪里？

读图 1→2→3

1. 先看层级（单元/集成/端到端）
2. 再看阶段门禁顺序
3. 最后看失败注入覆盖了哪些码

失败：任一阶段门禁红，不能进入下一阶段。

图例：绿色=已验证；橙色=推断；灰色=未验证 / 本机边界；蓝色=来源与去向。

关键元素：- ✅已验证：`--dry-run` 计数（`facts.md`，2026-10-27 本机）；S0–S7 门禁记录（`SETUP-FROM-SCRATCH.md` §7，Windows 2026-10-26 / Linux 2026-10-25）；旧系统计数未变（§5）；契约文件与 `compose.yaml` 的字段。- 🟡推断：`782 passed / 31 skipped`（README 与 HANDOFF 声称，本工作树未跑）；241 条写入总数（按 `plan.counts()` 公式推导，非 `--fresh` 实测）；`SETUP-FROM-SCRATCH.md` §3 的 `665 passed`（与 README 冲突，已过期）。- ⚪未验证：Docker 全栈（本工作树 `docker ps` 返回 `permission denied ... /var/run/docker.sock`）；真实 LLM 端点与真实校正通路；本机 `--fresh` 写三存储；本机前端构建与浏览器验收；真实召回接入 `/api/query`。- 去向：每张图的「证据」段 → `catalog-deploy-test.md` → 书稿 / PDF / QA 报告；未验证项必须在 QA 报告里保留。

证据：- `design/inventory/facts.md` §9：已验证 / 未验证清单（本设计工作树的共享底稿）。- `README.md` L25–L29：782 passed / 31 skipped、前端 40、SSE 夹具 14；验收前置。- `docs/DOCKER-HANDOFF.md` §5.1、§6：他机 782 passed / 31 skipped 与 `DOCKER SMOKE PASS` 记录。- `docs/SETUP-FROM-SCRATCH.md` §7 L1024–L1112：本机已验证 / 未在本 shell 端到端验证 / 明确边界。- `docs/SETUP-FROM-SCRATCH.md` §2.4 L320–L334：当前审计账户没有 Docker socket 权限。- 本次命令：`docker ps` → `DOCKER_DENIED`；`google-chrome` 存在；`python3 --version` = 3.12.3（渲染环境可用）。

边界：本图只描述本设计工作树的证据强度，不代表项目整体的验证状态。HANDOFF 记录 Docker smoke 在他机通过，但本工作树无法复跑，因此仍归入 ⚪；引用他机结果时必须在文字里写明来源，不能升级成 ✅。